オンプレミス構築 教育資料

# 構築手順 — Rust＋Topcoat版

OS だけが入った Windows Server の仮想マシンに、社内向けの Web アプリを建てる（PostgreSQL・IIS・Windows サービス・バックアップまで）

# 構築手順: Rust＋Topcoat版

## この資料について

この資料は、社内向けの Web アプリを、OS だけが入った仮想マシンに建てて、更新し続けるまでの手順を、**インフラの経験が無い人が、1つずつ手を動かして進められる**ように書いたものです。

- **前提:** 会社が仮想マシンを貸してくれて、**Windows Server（OS）だけ**を入れてくれる。データベースは、**仮想マシンの中に PostgreSQL を入れて**作る。
- **アプリ:** Rust と Topcoat で作った、1つの実行ファイル（この資料では `webapp.exe` と呼ぶ）。開発用のパソコンで作り、サーバーへ持ち込む。どんなアプリかは、00 基礎知識の「例にするアプリ」の表のとおり。
- **通信:** 社内では HTTPS を使えないので、**HTTP（80 番）**で動かす。HTTPS にするときの手順は、参考資料「HTTPS にする場合」に分けてある。
- **言葉:** この資料に出てくる言葉は、出てきた所で短く説明します。考え方から知りたいときは、**00 基礎知識**を先に読んでください。

## 記号と書き方の決まり

| 書き方 | 意味 |
|---|---|
| `webapp.example.local` | 本番の名前の**例**。本当の名前は、情報システム部門と決める |
| `webapp-stg.example.local` | 検証の名前の**例** |
| `example.com` | 会社のメールのドメインの**例** |
| `10.20.30.40` | サーバーの IP アドレスの**例** |
| （パスワード） | 自分で決めて、会社のパスワード管理の決まりに従って保管する値。この資料に本物を書き込まない |
| **設計例** | アプリの作り方で決まる所。この資料では、こう作る前提で書く。アプリの作者と合わせる |
| **確かめること** | コマンドを打ったあと、画面で見る所。結果の見た目は、OS の版や言語で少し違うので、資料には結果の写しを載せず、見る所だけを書く |

> **用語:** 設計例（この資料で「こう作る」と決めた一つの形。会社の決まりや、アプリの作り方によって変えてよい）

## この資料で使う版（2026-11-01 時点）

| もの | 版 | どこで使う |
|---|---|---|
| Windows Server | 2025 または 2022（会社が入れる） | サーバー |
| PostgreSQL | 18.6（Windows 用のインストーラー） | サーバー |
| IIS | 10（Windows Server に付いている） | サーバー |
| URL Rewrite | 2.1 | サーバー（IIS の追加の部品） |
| Application Request Routing（ARR） | 3.0 | サーバー（IIS の追加の部品） |
| WinSW | 2.12.0 | サーバー（実行ファイルを Windows サービスにする道具） |
| Rust | 1.99.1 | 開発用のパソコン |
| Topcoat | 0.11.0（topcoat CLI も 0.11.0） | 開発用のパソコン（アプリの中） |
| sqlx | 0.9.0（PostgreSQL につなぐ部品。設計例） | 開発用のパソコン（アプリの中） |

> **用語:** 版（バージョン：ソフトの何番目の出来かを表す番号。`18.6` なら「18 系の 6 番目の修正版」。同じ系の中の修正版は、安全の修正なので、上げてよいことが多い）

# 1. 全体像と役割分担

## 1.1 できあがりの形

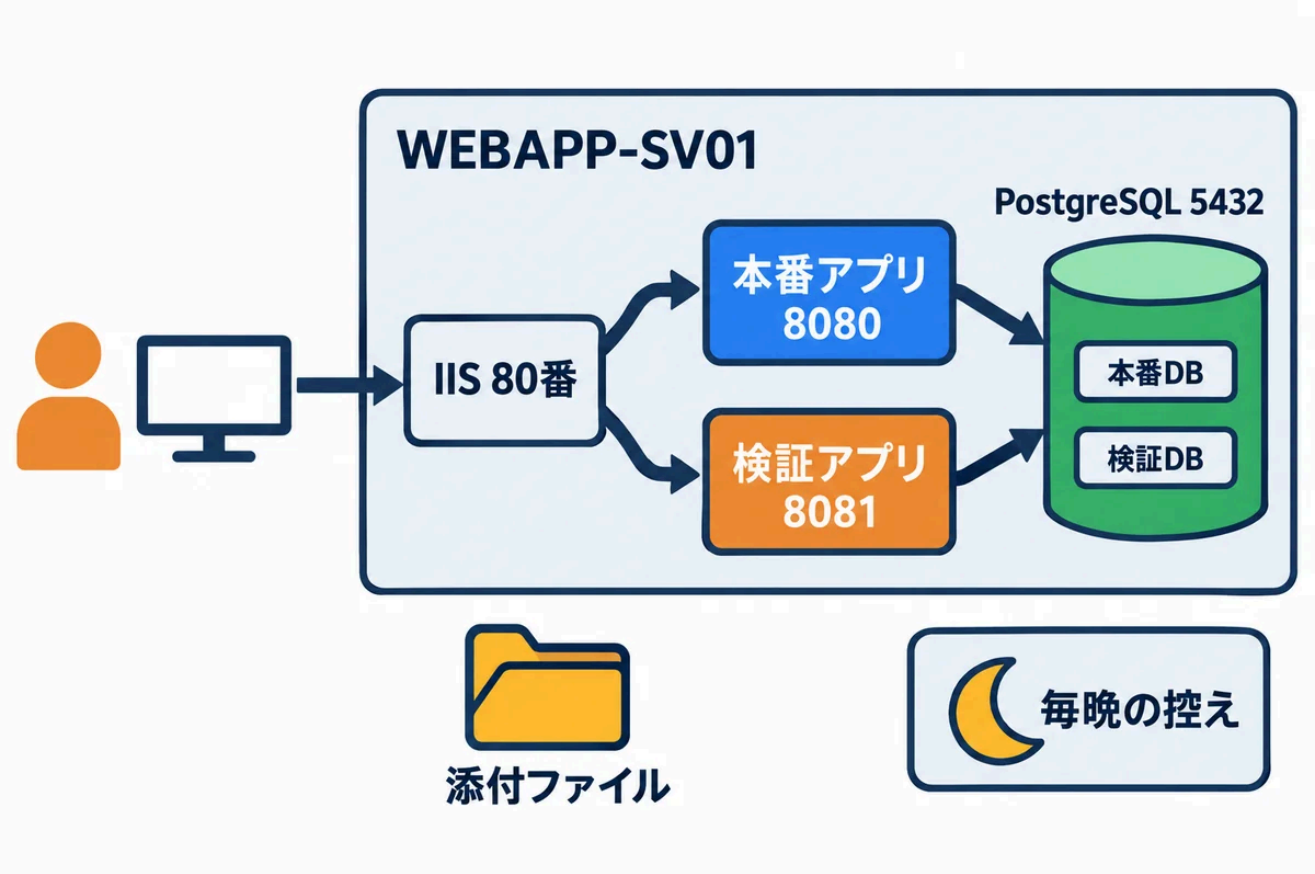



図1-1: この資料で建てる形。1台の仮想マシンに、本番と検証を同居させる（フォルダ・ポート・データベース・名前を分ける）

- 外から届くのは、IIS の **80 番**（と、管理者のパソコンからのリモート デスクトップの **3389 番**）だけ。
- アプリ（8080・8081）と PostgreSQL（5432）は **127.0.0.1** で待ち受けるので、サーバーの外からはつなげない。
- 添付ファイルは `D:\webapp\prod\files`（検証は `stg`）、毎晩 2:00 の控えは `D:\webapp\backup` と、別の場所（会社のファイルサーバーなど）に置く。
- メールは、Microsoft 365 に 25 番で渡す（10 章）。

## 1.2 本番と検証の分け方

同じサーバーに本番と検証を置き、次のように分けます。

| | 本番 | 検証 |
|---|---|---|
| 社員が開く名前（例） | `http://webapp.example.local/` | `http://webapp-stg.example.local/` |
| IIS のサイト | `webapp-prod` | `webapp-stg` |
| アプリのポート | `127.0.0.1:8080` | `127.0.0.1:8081` |
| Windows サービス | `webapp-prod` | `webapp-stg` |
| データベース | `webapp_prod`（ユーザー `webapp_prod_app`） | `webapp_stg`（ユーザー `webapp_stg_app`） |
| フォルダ | `D:\webapp\prod\` | `D:\webapp\stg\` |
| メール | 本当の宛先へ送る | **決まった宛先だけに送る**（設計例。検証から社員に本物の承認依頼が届かないように） |

> **用語:** 本番環境・検証環境（社員が実際に使う環境と、新しい版や設定を先に試す環境。00 基礎知識 9.4）

## 1.3 作業の順番

順番には理由があります。**下の物が無いと、上の物が動かない**順に建てます。

1. サーバーを受け取って、最初の確認をする（3 章）
2. フォルダと、アプリを動かすアカウントの考え方を決める（4 章）
3. データベースを入れる（5 章）。アプリは、データベースが無いと起動できないため
4. アプリを持ち込み、Windows サービスにして、サーバーの中で動くことを確かめる（6・7 章）
5. 受付（IIS）を設定して、社員のパソコンから開けるようにする（8 章）
6. ファイアウォールを確かめる（9 章）
7. メールを送れるようにする（10 章）
8. 毎晩のバックアップを設定し、戻せることを確かめる（11 章）
9. 定期処理・ログ・監視を整える（12 章）
10. 検証で一通り確かめてから、本番を始める（13 章）

**検証環境から先に建てて、うまくいったら同じことを本番にする**と、失敗しても社員に迷惑がかかりません。この資料の例は本番で書いていますが、`prod` を `stg` に、`8080` を `8081` に読みかえれば検証になります。

# 2. 作業の前に決めること・頼むこと

## 2.1 情報システム部門に頼むこと




図2-1: 会社の設備にかかわる所は、早めに頼む

インフラの作業のうち、**会社の設備や決まりにかかわる所**は、自分ではできません。早めに頼みます。依頼の文面の例は、**07 付録**の「依頼シート」にまとめます。

| 頼むこと | なぜ要るか | この資料 |
| --- | --- | --- |
| 仮想マシン（Windows Server・デスクトップ エクスペリエンス）と、その大きさ | アプリを動かす建物 | 2.2 |
| 固定の IP アドレスと、サーバーの名前 | 社員が毎回同じ所につなぐため | 00 の 4 章 |
| DNS に、本番と検証の名前を登録 | `webapp.example.local` で開けるようにするため | 8 章 |
| ネットワークの関所で、社内から 80 番を通す。外向きは Microsoft 365 への 25 番だけを通す | 社員が開けるように。メールが出せるように | 9 章 |
| Microsoft 365 のコネクタ、送信元アドレス、SPF | アプリのメールを受け付けてもらうため | 10 章 |
| 会社のインターネットの出口の IP アドレスが固定か、の確認 | コネクタの条件 | 10 章 |
| バックアップを写す、別の場所（ファイルサーバーの共有フォルダなど） | サーバーが壊れても控えを残すため | 11 章 |
| サーバーの管理者の権限と、リモート デスクトップでの入り方 | 自分で作業するため | 3 章 |
| Windows Update とウイルス対策の、会社の決まり | 再起動の時間、除外の設定などを合わせるため | 14 章 |
| 社員一覧の CSV（人事など） | 初期データ（社員データ）のため | 7.7 |

## 2.2 仮想マシンの大きさの目安

大きさは、**使う人数**と**保存するファイルの量**で決まります。下は、考え方と、例の数字です。自分の会社の数字で計算し直してください。

| 項目 | 目安（例） | 考え方 |
| --- | --- | --- |
| vCPU | 4 | Windows Server・IIS・PostgreSQL・アプリ（本番と検証）が同時に動く。全社員の業務アプリとしては、最初は 4 で足りることが多い |
| メモリ | 8GB 以上（できれば 16GB） | Windows Server だけで数 GB を使う。PostgreSQL はメモリが多いほど速い |
| C ドライブ | 100GB | OS と、Windows Update のための空き |
| D ドライブ | 200GB（下の計算で決める） | データベース・添付ファイル・バックアップ・ログ |

D ドライブの計算の例（数字はすべて仮）：

| 項目 | 例の数字 | 5 年分 |
|---|---|---|
| 添付ファイル（1 件の依頼に平均 10MB、年 1,000 件） | 年 10GB | 50GB（5 年保存する場合） |
| データベース（図面の JSON を含む） | 年 2GB | 10GB |
| 添付ファイルのバックアップ（11 章の方式では、本体と同じ量） | — | 50GB |
| データベースのバックアップ（14 日分） | 1 回 数 GB × 14 | 数十 GB |
| 合計の目安 | | 150GB 前後 → 余裕を見て 200GB |

> **用語:** デスクトップ エクスペリエンス（Windows Server を、画面（デスクトップ）付きで入れる形。画面の無い Server Core より、初めての人に分かりやすい。00 基礎知識 3.1）

## 3. サーバーに入る・最初の確認

### 3.1 リモート デスクトップで入る

サーバーは、データセンターの中にあり、画面もキーボードもありません。自分のパソコンから、**リモート デスクトップ**でサーバーの画面を開いて操作します。

1. 自分のパソコンで「リモート デスクトップ接続」（`mstsc`）を開きます。
2. 「コンピューター」に、サーバーの名前か IP アドレス（例：`10.20.30.40`）を入れて「接続」を押します。
3. 情報システム部門から教わったアカウント（ふつうは会社のアカウント）でサインインします。
4. 初めてつなぐときに「証明書の警告」が出ることがあります。これはリモート デスクトップ用の証明書の話で、アプリの証明書とは別です。情報システム部門に、出てよいものかを確かめます。

> **用語:** リモート デスクトップ（RDP：ネットワークの向こうのコンピューターの画面を、自分のパソコンに映して操作する仕組み。3389 番ポートを使う）

**注意:** リモート デスクトップの窓を「×」で閉じても、サーバーの中では、自分のサインインが続いています（切断）。作業が終わったら、スタートメニューから「サインアウト」します。

### 3.2 管理者の PowerShell を開く

この資料のコマンドは、**管理者として実行した PowerShell** で打ちます。

1. スタートボタンを右クリックします。
2. 「ターミナル（管理者）」または「Windows PowerShell（管理者）」を選びます（版によって名前が違います）。
3. 窓の題名に「管理者」と出ていることを確かめます。

> **用語:** PowerShell（Windows を文字のコマンドで操作する道具。00 基礎知識 3.5）

## 3.3 受け取ったサーバーを確かめる

受け取ったら、まず「頼んだとおりか」を確かめて、記録に残します（07 付録のチェック表）。

```
# OS の名前と版
Get-ComputerInfo -Property OsName, OsVersion, CsName, CsDomain

# IP アドレス・DNS サーバー・デフォルトゲートウェイ
Get-NetIPConfiguration

# ディスク（C と D があるか、大きさと空き）
Get-Volume | Where-Object DriveLetter | Format-Table DriveLetter, FileSystem, Size, SizeRemaining

# CPU の数とメモリ
Get-CimInstance Win32_ComputerSystem | Format-List NumberOfLogicalProcessors, TotalPhysicalMemory

# 時刻の合わせ先（会社のドメインに参加していれば、ドメインのサーバーに合わせている）
w32tm /query /status
```

**確かめること:**

- OS の名前が、頼んだ版（例：Windows Server 2025 Standard）になっている。
- `CsDomain` が、会社のドメインになっている（ドメインに参加している）。
- IP アドレスが、頼んだ固定の IP アドレスになっている。
- D ドライブがあり、`FileSystem` が `NTFS` になっている。
- `w32tm` の「ソース」が、会社の時刻のサーバーになっている。時刻がずれていると、ログの時刻・承認 URL の有効期限・毎朝 8:00 の処理がずれる。

> **用語:** NTFS（エヌティーエフエス：Windows のディスクの形式。フォルダやファイルごとに、誰が読めて誰が書けるかを決められる）

## 3.4 時刻のゾーンを日本時間にする

例のアプリは日本時間で動くので、サーバーの時刻のゾーンを日本にします。

```
# 今の設定を見る
Get-TimeZone

# 日本時間にする（すでに日本時間なら不要）
Set-TimeZone -Id "Tokyo Standard Time"
```

> **用語:** タイムゾーン（地域ごとの時刻のずれの決まり。日本は「協定世界時（UTC）＋9 時間」。Windows では `Tokyo Standard Time` という名前）

### 3.5 Windows Update

建てる前に、Windows Update を済ませて、再起動しておきます。会社によっては、Windows Update を情報システム部門がまとめて管理しています（WSUS など）。**自分で更新してよいか、更新と再起動の時間の決まりを、先に確かめます。**

> **用語:** WSUS（ダブリューサス。Windows Server Update Services：会社の中で、Windows Update の配り方をまとめて管理する仕組み）

## 4. フォルダとアカウントの準備

### 4.1 フォルダの設計

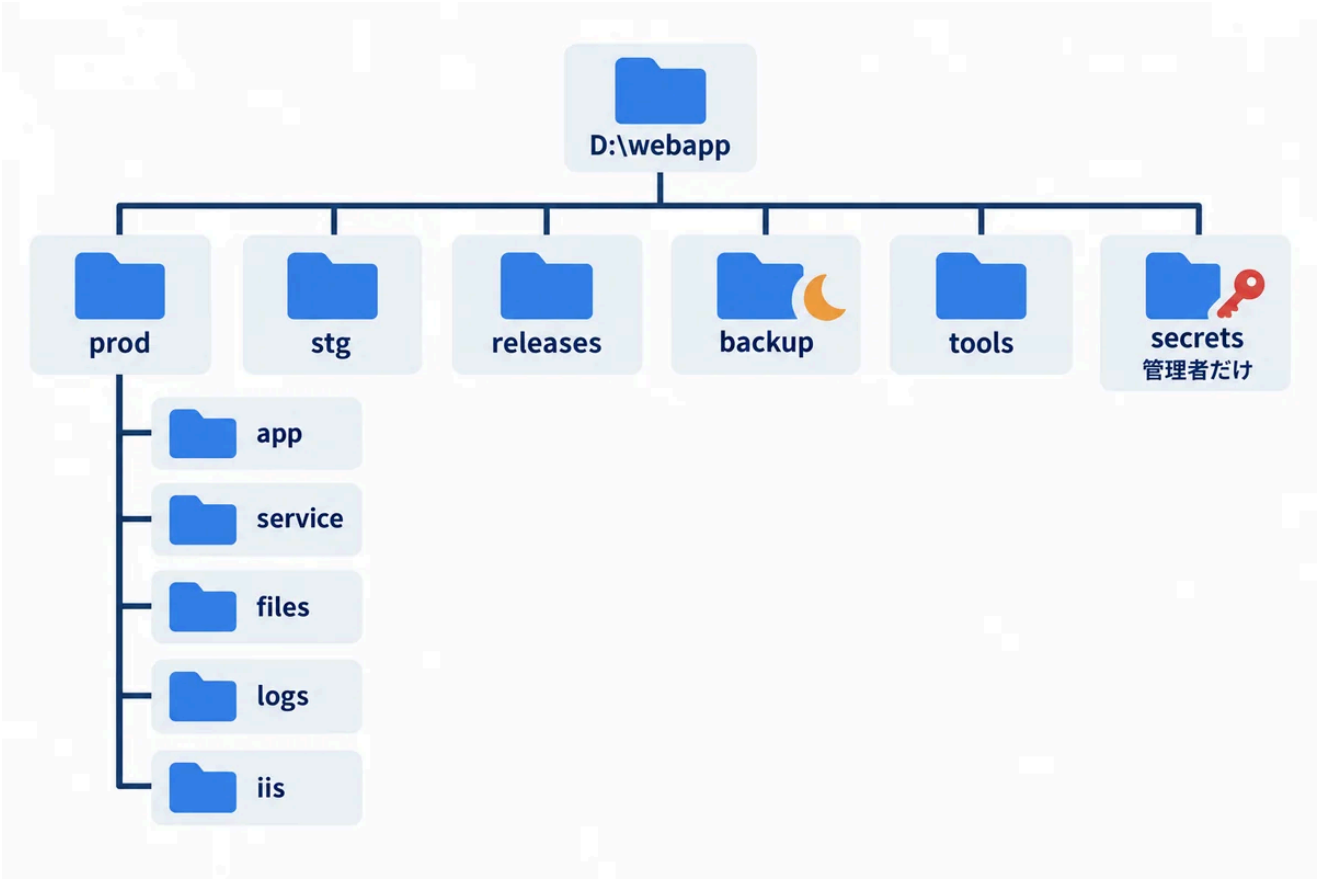



図4-1: フォルダの配置。本番と検証で同じ形にする

データとアプリは、**D ドライブ**にまとめます。OS（C ドライブ）と分けておくと、ディスクの拡張や、OS の入れ直しがしやすくなります。

```
D:\webapp\
├─ prod\              本番
│  ├─ app\           アプリ（webapp.exe と assets\）。更新のときに入れ替える所
│  ├─ service\       Windows サービスにする道具（WinSW）と、その設定ファイル
│  ├─ files\         添付ファイルの保存フォルダ（JPG・PNG・PDF）
│  ├─ logs\          アプリのログ
│  └─ iis\           IIS のサイトの置き場（web.config だけを置く）
├─ stg\              検証（prod と同じ並び）
├─ releases\         受け取ったアプリの版（zip）。切り戻しのために残しておく
├─ backup\           毎晩の控え（db\・files\・logs\）
├─ tools\            バックアップなどの道具（PowerShell のスクリプト）
├─ secrets\          データベースのパスワードのファイル（管理者と SYSTEM だけが読める）
└─ installers\       持ち込んだインストーラー
D:\pgsql\18\data\    PostgreSQL のデータ（5 章で、インストールのときに指定する）
```

まとめて作ります。

```powershell
$dirs = @(
  "D:\webapp\prod\app", "D:\webapp\prod\service", "D:\webapp\prod\files", "D:\webapp\prod\logs", "D:\webapp\prod\iis",
  "D:\webapp\stg\app",  "D:\webapp\stg\service",  "D:\webapp\stg\files",  "D:\webapp\stg\logs",  "D:\webapp\stg\iis",
  "D:\webapp\releases", "D:\webapp\backup\db", "D:\webapp\backup\files", "D:\webapp\backup\logs",
  "D:\webapp\tools", "D:\webapp\secrets", "D:\webapp\installers"
)
foreach ($d in $dirs) { New-Item -ItemType Directory -Force -Path $d | Out-Null }
```

> **用語:** スクリプト（コマンドを順に並べて、ファイルにしたもの。`.ps1` は PowerShell のスクリプト）

## 4.2 フォルダのアクセス許可を絞る

D ドライブの一番上には、ふつう「Users（サーバーにサインインできる全員）が読める」という許可が付いています。`D:\webapp` には、パスワードを書いたファイルや、社員の添付ファイルが入るので、**管理者と SYSTEM だけ**にします。

```powershell
# 上（D:\）からの許可の引き継ぎを切り、管理者と SYSTEM だけにする
#   *S-1-5-32-544 は Administrators（管理者のグループ）、*S-1-5-18 は SYSTEM（OS 自身）
#   名前ではなく番号（SID）で書くと、日本語の Windows でも英語の Windows でも同じに動く
icacls D:\webapp /inheritance:r /grant:r "*S-1-5-32-544:(OI)(CI)F" "*S-1-5-18:(OI)(CI)F"

# 結果を見る
icacls D:\webapp
```

**確かめること:** 結果に、`BUILTIN\Administrators:(OI)(CI)(F)` と `NT AUTHORITY\SYSTEM:(OI)(CI)(F)` の2つだけが出ている（日本語の Windows でも、この2つの名前は英語で出ることが多い）。

> **用語:** アクセス許可（ACL：フォルダやファイルごとの「誰が何をしてよいか」の一覧。`F` は何でもできる、`M` は変更できる、`RX` は読んで実行できる）

> **用語:** 継承（引き継ぎ：上のフォルダの許可が、中のフォルダとファイルに自動で付く仕組み。`(OI)(CI)` は「中のファイルとフォルダにも引き継ぐ」の印）

> **用語:** SID（エスアイディー：アカウントやグループに付いている、世界で1つの番号。名前が言語で変わっても、番号は変わらない）

## 4.3 アプリを動かすアカウント（仮想アカウント）

アプリを、管理者や自分のアカウントで動かしてはいけません。乗っ取られたときに、サーバーの全部をさわられてしまうからです（**最小権限**。00 基礎知識 10 章）。

この資料では、Windows の**仮想アカウント**を使います。仮想アカウントは、サービスを登録すると自動でできる、そのサービス専用のアカウントです（名前は `NT SERVICE\サービスの名前`）。

- パスワードが無いので、パスワードの期限切れでサービスが止まることが無い。
- 管理者の権限を持たない。許可したフォルダだけをさわれる。
- サービスごとに別のアカウントになるので、本番のアプリ（`NT SERVICE\webapp-prod`）は、検証のフォルダをさわれない。

| アカウント | 何をするか | 許可する所 |
|---|---|---|
| `NT SERVICE\webapp-prod` | 本番のアプリを動かす | `D:\webapp\prod` を読む・実行する。`files\` と `logs\` は書ける |
| `NT SERVICE\webapp-stg` | 検証のアプリを動かす | `D:\webapp\stg` を同じように |
| `SYSTEM` | 毎晩のバックアップ（タスク スケジューラ） | `D:\webapp` の全部（4.2 で付けた） |

仮想アカウントは、**サービスを登録したあと**でないと許可を付けられません。許可の付け方は、7.4 で、サービスを登録したあとに行います。

> **用語:** 仮想アカウント（Virtual Account：サービス1つに1つできる、パスワードの無い専用のアカウント。`NT SERVICE\サービス名` という名前。会社の決まりで、ドメインのサービス用アカウントや gMSA を使うよう決まっている場合は、そちらに合わせる）

> **用語:** gMSA（ジーエムエスエー。グループ管理サービス アカウント：パスワードを Active Directory が自動で管理する、サービス用のドメインのアカウント。作るのは情報システム部門）

## 5. データベース（PostgreSQL）

> **用語:** PostgreSQL（無料で使えるデータベース。00 基礎知識 7 章）

### 5.1 インストーラーを持ち込む

サーバーはインターネットにつながらない前提なので、**開発用のパソコンで**インストーラーを手に入れて、サーバーへ持ち込みます。

1. 開発用のパソコンで、PostgreSQL の公式サイトのダウンロードのページ（`https://www.postgresql.org/download/windows/`）から、EDB 社の Windows 用インストーラーのページへ進みます。
2. 「18.6」の「Windows x86-64」をダウンロードします（`postgresql-18.6-1-windows-x64.exe` のような名前）。
3. サーバーの `D:\webapp\installers\` へコピーします。

> **用語:** インストーラー（ソフトをコンピューターに入れるためのプログラム）

> **用語:** x86-64（エックスはちろく・x64：今の多くのサーバーとパソコンの CPU の種類。サーバーが x64 なら、x64 用を選ぶ）

**持ち込むときの確かめ:** 壊れずにコピーできたかを、両方のコンピューターで「ハッシュ値」を出して比べます。2つの値が同じなら、中身は同じです。

```
Get-FileHash D:\webapp\installers\postgresql-18.6-1-windows-x64.exe -Algorithm SHA256
```

> **用語:** ハッシュ値（ファイルの中身から計算した、指紋のような長い文字列。1 文字でも中身が違うと、まったく違う値になる。SHA256 はその計算のしかたの1つ）

### 5.2 インストールする

インストーラーをダブルクリックして、画面の質問に答えます。

| 画面 | 選ぶもの | 理由 |
| --- | --- | --- |
| Installation Directory | そのまま（`C:\Program Files\PostgreSQL\18`） | プログラム本体は C ドライブでよい |
| Select Components | PostgreSQL Server・pgAdmin 4・Command Line Tools に印。**Stack Builder は外す** | Stack Builder は、インターネットから追加の部品を取る道具で、今回は使わない |
| Data Directory | `D:\pgsql\18\data` | データは D ドライブに置く（4.1） |
| Password | スーパーユーザー `postgres` のパスワード（長く、推測できないもの） | データベースの何でもできるアカウント。**会社のパスワード管理の決まりに従って保管する** |
| Port | `5432`（そのまま） | PostgreSQL の決まりの番号 |
| Advanced Options（Locale） | そのまま（Default locale） | 文字の並べ方などの決まり。そのままでよい |
| Pre Installation Summary | 内容を確かめて「Next」 | |

インストールが終わると、PostgreSQL が **Windows サービス**として動き始めます。

```
Get-Service postgresql*
```

**確かめること:** `postgresql-x64-18` という名前のサービスが `Running`（動いている）になっている。

> **用語:** スーパーユーザー（データベースの中で、何でもできるユーザー。PostgreSQL では `postgres`。アプリには使わせない）

> **用語:** pgAdmin（ピージーアドミン：PostgreSQL を画面で操作する道具。表の中身を見るときなどに便利）

## 5.3 サーバーの中からだけつなげるようにする

PostgreSQL の Windows 用インストーラーは、**ほかのコンピューターからもつなげる**設定（`listen_addresses = '*'`）で入ることがあります。今回は、アプリと同じサーバーの中からしか使わないので、**サーバーの中だけ**（`localhost`）にします。

1. 設定ファイル `D:\pgsql\18\data\postgresql.conf` を、管理者のメモ帳で開きます。

```
powershell notepad D:\pgsql\18\data\postgresql.conf
```

2. `listen_addresses` の行を探して、次のようにします（行の先頭に `#` があれば消します）。

```
text listen_addresses = 'localhost'
```

3. ついでに、次の3つを確かめます（無ければ書き足します）。

`text timezone = 'Asia/Tokyo' log_timezone = 'Asia/Tokyo' lc_messages = 'C'`

`lc_messages = 'C'` は、PostgreSQL のメッセージを英語にする設定です。日本語の Windows で既定のまま入れると、メッセージが日本語になりますが、**ログのファイルに、UTF-8 の行と Shift_JIS の行が混ざって**、どちらの文字コードで開いても一部が文字化けします（00 基礎知識 0.4。PostgreSQL 18.3 の日本語ロケールで試して確かめました）。英語にすると、ログが英数字だけになって化けず、エラーの文をそのまま検索して調べられます。

4. 保存して、PostgreSQL を再起動します。

`powershell Restart-Service postgresql-x64-18`

5. 待ち受けている住所を確かめます。

`powershell Get-NetTCPConnection -State Listen -LocalPort 5432 | Format-Table LocalAddress, LocalPort, OwningProcess`

**確かめること:** `LocalAddress` が `127.0.0.1` と `::1` だけになっている（`0.0.0.0` や `::` があれば、まだ外からも待ち受けている）。

> **用語:** listen_addresses（PostgreSQL が、どの住所で待ち受けるかの設定。`localhost` なら、このサーバーの中からだけ）

> **用語:** `::1`（IPv6 という新しい形の住所での、localhost。`127.0.0.1` と同じ意味）

もう1つの関所の設定ファイル `pg_hba.conf` は、「どこから、どのユーザーが、どの方法でログインしてよいか」を決めます。インストーラーが作る形では、`127.0.0.1/32` と `::1/128` から、パスワード（`scram-sha-256`）でつなぐ行が入っています。中を見て、**`0.0.0.0/0` や `all` の住所の行が無い**ことを確かめます。

```
Get-Content D:\pgsql\18\data\pg_hba.conf | Select-String -Pattern "^\s*host"
```

> **用語:** pg_hba.conf（PostgreSQL の「誰がどこからつないでよいか」の決まりを書いたファイル。hba は host-based authentication）

> **用語:** scram-sha-256（パスワードを、そのまま送らずに確かめる方式。今の PostgreSQL の決まりの方式）

## 5.4 psql を使えるようにする

`psql` は、PostgreSQL に SQL を送る、文字の道具です。インストールした場所の `bin` にありますが、そのままでは PowerShell から呼べないので、その窓の中だけ、場所を教えます。

```
$env:Path += ";C:\Program Files\PostgreSQL\18\bin"
psql --version
```

> **用語:** psql（ピーエスキューエル：PostgreSQL に SQL を打ち込む、文字の道具）

> **用語:** PATH（パス：コマンドの名前だけで呼べるように、プログラムを探す場所を並べた環境変数）

## 5.5 本番と検証のデータベースとユーザーを作る

スーパーユーザーでつなぎます（5.2 で決めたパスワードを聞かれます）。

```
psql -h 127.0.0.1 -U postgres
```

`postgres=#` と出たら、PostgreSQL の中です。次の SQL を、1 行ずつ打ちます。

```
-- 本番のアプリ用のユーザー（ログインできるロール）を作る
CREATE ROLE webapp_prod_app LOGIN;
-- パスワードを付ける（画面に出さずに2回聞かれる）
\password webapp_prod_app

-- 本番のデータベースを作り、持ち主をアプリ用のユーザーにする
CREATE DATABASE webapp_prod OWNER webapp_prod_app ENCODING 'UTF8' TEMPLATE template0;
-- ほかのユーザーからは、つなげないようにする
REVOKE ALL ON DATABASE webapp_prod FROM PUBLIC;

-- 検証も同じように作る
CREATE ROLE webapp_stg_app LOGIN;
\password webapp_stg_app
CREATE DATABASE webapp_stg OWNER webapp_stg_app ENCODING 'UTF8' TEMPLATE template0;
REVOKE ALL ON DATABASE webapp_stg FROM PUBLIC;

-- できたものを見る
\l webapp*
\du webapp*

-- psql を終わる
\q
```

- **本番と検証で、ユーザーもパスワードも分けます。**検証の設定を間違えても、本番のデータベースにつながらないようにするためです。

- パスワードは、`CREATE ROLE ... PASSWORD '...'` と SQL に書くこともできますが、打った文が記録に残ることがあるので、`\password` を使います。

> **用語:** ロール（PostgreSQL のユーザーとグループのこと。`LOGIN` を付けたロールが、ログインできるユーザー）

> **用語:** OWNER（持ち主：そのデータベースの中で、テーブルを作ったり消したりできるユーザー。アプリがテーブルを作る（マイグレーション。00 基礎知識 7.3）ので、アプリ用のユーザーを持ち主にする）

> **用語:** UTF8（ユーティーエフエイト：日本語を含む、世界中の文字を表せる文字の決まり。データベースは UTF8 で作る）

> **用語:** PUBLIC（PostgreSQL の、全部のユーザーを表す特別な名前。`REVOKE ... FROM PUBLIC` は「全員から取り上げる」）

## 5.6 アプリ用のユーザーでつなげるか確かめる

```
psql -h 127.0.0.1 -U webapp_prod_app -d webapp_prod -c "SELECT current_user, current_database(), now();"
psql -h 127.0.0.1 -U webapp_stg_app  -d webapp_prod -c "SELECT 1;"
```

**確かめること:**

- 1 つ目は、`webapp_prod_app`・`webapp_prod`・今の日本時間（`+09` が付く）が出る。
- 2 つ目（検証のユーザーで、本番のデータベースにつなぐ）は、**つなげずにエラーになる**（「permission denied」や「権限がありません」）。これが正しい結果です。

## 5.7 テーブルを作る

テーブルは、手で作りません。**アプリが、マイグレーション（番号付きのテーブル変更の手順）で作ります**（7.6）。手で作ると、アプリの版とテーブルの形がずれるからです。

> **設計例:** アプリは sqlx（0.9.0）のマイグレーションの機能を使い、起動のときに、まだ当てていない手順を当てる。

## 5.8 SQL Server を使うよう言われたら

会社の決まりで SQL Server を使う場合は、この章の代わりに SQL Server を入れます。考え方は同じですが、道具と、アプリの部品が変わります（00 基礎知識 7.5 の表）。**アプリを作り始める前に**決めてください。

# 6. アプリを作る（開発用のパソコン）と持ち込む




図6-1: 開発用のパソコンで作り、まとめ、指紋（ハッシュ値）を控えて持ち込む

## 6.1 本番用の実行ファイルを作る

開発用のパソコンの、アプリのフォルダで打ちます。

```
# Cargo.lock のとおりの版で、本番用（release）に作る
cargo build --release --locked

# 実行ファイルの隣に、CSS などの束（assets\）を作る
topcoat asset bundle --release
```

- できるのは、`target\release\webapp.exe` と、その隣の `target\release\assets\` の2つです。**2つを必ず一緒に**持ち込みます。Topcoat のアプリは、起動したときに「自分の隣の `assets\` 」を読むので、`assets\` が無いと起動しません。
- 画面で使う HTML や JavaScript（図面を描く道具など）を、アプリの中に入れて配信する作りなら、それも実行ファイルか `assets\` に入っています。別のフォルダにしている場合は、そのフォルダも持ち込みます（アプリの作り方で決まる）。

> **用語**: release ビルド（リリースビルド：本番用に、速く動くように作ること。開発中の debug ビルドより、作るのに時間がかかるが、動きは速い）

> **用語**: Cargo.lock（カーゴロック：アプリが使う部品の、正確な版を記録したファイル。`--locked` を付けると、このファイルのとおりの版で作る。「部品の版を固定する」は、このファイルで守る）

## 6.2 サーバーに無い部品に頼らないようにする

Rust で作った Windows の実行ファイルは、そのままだと、**Visual C++ の再頒布可能パッケージ**（`VCRUNTIME140.dll` など）がサーバーに入っていることを前提にします。入っていないサーバーでは、起動したとたんに「DLL が見つかりません」で止まります。

アプリのフォルダの `.cargo\config.toml` に、次を書いておくと、その部品を実行ファイルの中に入れて作れます（静的リンク）。

```
# .cargo/config.toml（アプリのフォルダの中）
[target.x86_64-pc-windows-msvc]
rustflags = ["-C", "target-feature=+crt-static"]
```

> **用語**: DLL（ディーエルエル：ほかのプログラムが使う部品を入れたファイル。実行ファイルとは別に、OS の中に入っている必要がある）

> **用語**: 静的リンク（部品を、実行ファイルの中に一緒に入れてしまうこと。サーバーに部品が無くても動く）

**開発用のパソコンが ARM（Snapdragon など）の場合**: サーバーは x64 なので、`rustup target add x86_64-pc-windows-msvc` をして、`cargo build --release --locked --target x86_64-pc-windows-msvc` で作ります。できる場所は `target\x86_64-pc-windows-msvc\release\` です。`topcoat asset bundle` には `--target` が無いので、`--out` で束の置き場を指定して作り、x64 の実行ファイルの隣に置きます。CSS の束は、CPU の種類によらず同じ中身です。**検証環境で起動して確かめてから**、本番に使います。

## 6.3 版の名前を付けて、まとめる

持ち込む物は、**版の番号を付けた zip** にまとめます。あとで「今どの版が動いているか」「1つ前の版はどれか」が分かるようにするためです。

```powershell
# 開発用のパソコンで（例：版 1.0.0）
$ver = "1.0.0"
$out = "dist\webapp-$ver"
New-Item -ItemType Directory -Force $out | Out-Null
Copy-Item target\release\webapp.exe $out
Copy-Item target\release\assets $out -Recurse
Compress-Archive -Path "$out\*" -DestinationPath "dist\webapp-$ver.zip" -Force
Get-FileHash "dist\webapp-$ver.zip" -Algorithm SHA256
```

- zip は、サーバーの `D:\webapp\releases\` にコピーします。コピーの方法（会社のファイルサーバー経由、リモート デスクトップのドライブの共有など）は、会社の決まりに合わせます。
- サーバーでも `Get-FileHash` を打って、ハッシュ値が同じことを確かめます（5.1 と同じ）。
- **パスワードなどの秘密は、zip に入れません。**秘密は、サーバーの上で設定します（7.3）。

> **用語:** zip（ジップ：たくさんのファイルを1つにまとめて、小さくしたファイル）

## 6.4 HTTP で動かすための、アプリの設定（セッションのクッキー）




図6-2: HTTP では、Secure の付いたクッキーは受け取られない

**これをしないと、HTTP ではログインが続きません。**アプリを作るときに、先に入れておきます。

Topcoat 0.11 の既定のセッション（`SessionConfig::default()`）は、ログインの印を運ぶクッキーに、**`__Host-` の名前と `Secure` の印を必ず付けます**（topcoat-session 0.11.0 のソースで確かめました）。ブラウザは、HTTP のページから届いた `Secure` のクッキーを受け取らないので、ログインしても、次の画面でログインしていない扱いになります。

そこで、Topcoat が用意している「クッキーの運び方を差しかえる」入口（`TokenStore`）を使い、`Secure` と `__Host-` だけを外したクッキーにします。ほかの守り（`HttpOnly`・`SameSite=Lax`・`Path=/`）は残します。

> **用語:** `Secure`（セキュア：クッキーに付ける印の1つ。「HTTPS のときだけ送る・受け取る」の意味。HTTP では、この印の付いたクッキーは使えない）

> **用語:** `__Host-`（ホスト プレフィックス：クッキーの名前の頭に付ける決まりの文字。付けると、ブラウザは `Secure` などの条件を満たすときだけ受け取る）

> **用語:** `HttpOnly`（JavaScript からクッキーを読めないようにする印）・`SameSite=Lax`（ほかのサイトからの POST に、クッキーを付けないようにする印）

アプリのソースに、次のファイルを足します（`src/http_cookie.rs`）。

```rust
//! HTTP（暗号化なし）の社内環境で使う、セッションのクッキーの運び方。
//!
//! Topcoat 0.11 の既定（`CookieTokenStore`）は、クッキーに `__Host-` の名前と
//! `Secure` を必ず付ける。ブラウザは HTTP のページでは `Secure` のクッキーを
//! 受け取らないので、HTTP ではログインが続かない。
//! ここでは `Secure` と `__Host-` だけを外し、`HttpOnly`・`SameSite=Lax`・`Path=/` は残す。
//! HTTPS にしたら、このファイルは使わず、決まりのクッキーに戻す（期限の 60 分は残す）。

use std::time::Duration;

use topcoat::{
    context::Cx,
    cookie::{Cookie, Cookies, SameSite, time},
    session::{Token, TokenStore, TokenStoreFuture},
};

/// セッションのクッキーの名前。
const NAME: &str = "session";

/// HTTP 用のセッションのクッキー（`Secure` なし）。
pub struct HttpCookieTokenStore;

fn cookies(cx: &Cx) -> impl Cookies {
    topcoat::cookie::cookies(cx)
        .override_same_site(SameSite::Lax)
        .override_http_only(true)
        .override_secure(false)
        .override_path("/")
}

impl TokenStore for HttpCookieTokenStore {
    fn read<'a>(&'a self, cx: &'a Cx) -> TokenStoreFuture<'a, Option<Token>> {
        Box::pin(async move {
            let Some(cookie) = cookies(cx).get(NAME) else {
                return Ok(None);
            };
            Ok(Token::decode(cookie.value_trimmed()).ok())
        })
    }

    fn write<'a>(
        &'a self,
        cx: &'a Cx,
        token: Token,
        max_age: Duration,
    ) -> TokenStoreFuture<'a, ()> {
        Box::pin(async move {
            let max_age = time::Duration::try_from(max_age)?;
            cookies(cx)
                .override_max_age(max_age)
```

```
                .add(Cookie::new(NAME, token.encode()));
            Ok(())
        })
    }

    fn delete<'a>(&'a self, cx: &'a Cx) -> TokenStoreFuture<'a, ()> {
        Box::pin(async move {
            cookies(cx).remove(Cookie::new(NAME, ""));
            Ok(())
        })
    }
}
```

ルーターを組み立てる所で、既定の `SessionConfig::default()` の代わりに、これを使います。60 分の期限も、ここで決めます。

```
// 例：ルーターを組み立てる所
.cookies()
.sessions(
    SessionConfig::builder()
        .token_store(crate::http_cookie::HttpCookieTokenStore)
        .lifetime(std::time::Duration::from_secs(60 * 60))
        .build(),
)
```

`src/lib.rs`（または `main.rs`）に `pub mod http_cookie;` を足すのも忘れないでください。

**確かめたこと（2026-11-01）**：Topcoat 0.11.0 のアプリ（Rust の教材の RustNote）を、この形に直して HTTP で動かしました。

- ログインの返事のクッキーが、`session=…; HttpOnly; SameSite=Lax; Path=/; Max-Age=3600` になった（`Secure` と `__Host-` が無い）。
- そのクッキーを付けて、ログインが要る画面を開くと `200` になった。
- `lifetime` は「使われないまま、この時間がたつと切れる」期限です。操作のたびに延びるかどうかは、アプリでセッションをどう扱うかによります。60 分で自動でログアウトされるかを、検証環境で確かめます。
- HTTPS にしたら、このファイルは使わず、`.token_store(...)` の行を消して、Topcoat の決まりのクッキーに戻します（参考資料「HTTPS にする場合」の 7 章）。**`SessionConfig::default()` にはしません。**決まりの期限は 30 日なので、60 分で自動でログアウトしなくなります（topcoat-session 0.11.0 のソースで確かめました）。

# 7. アプリを Windows サービスにする

## 7.1 なぜ WinSW を使うのか

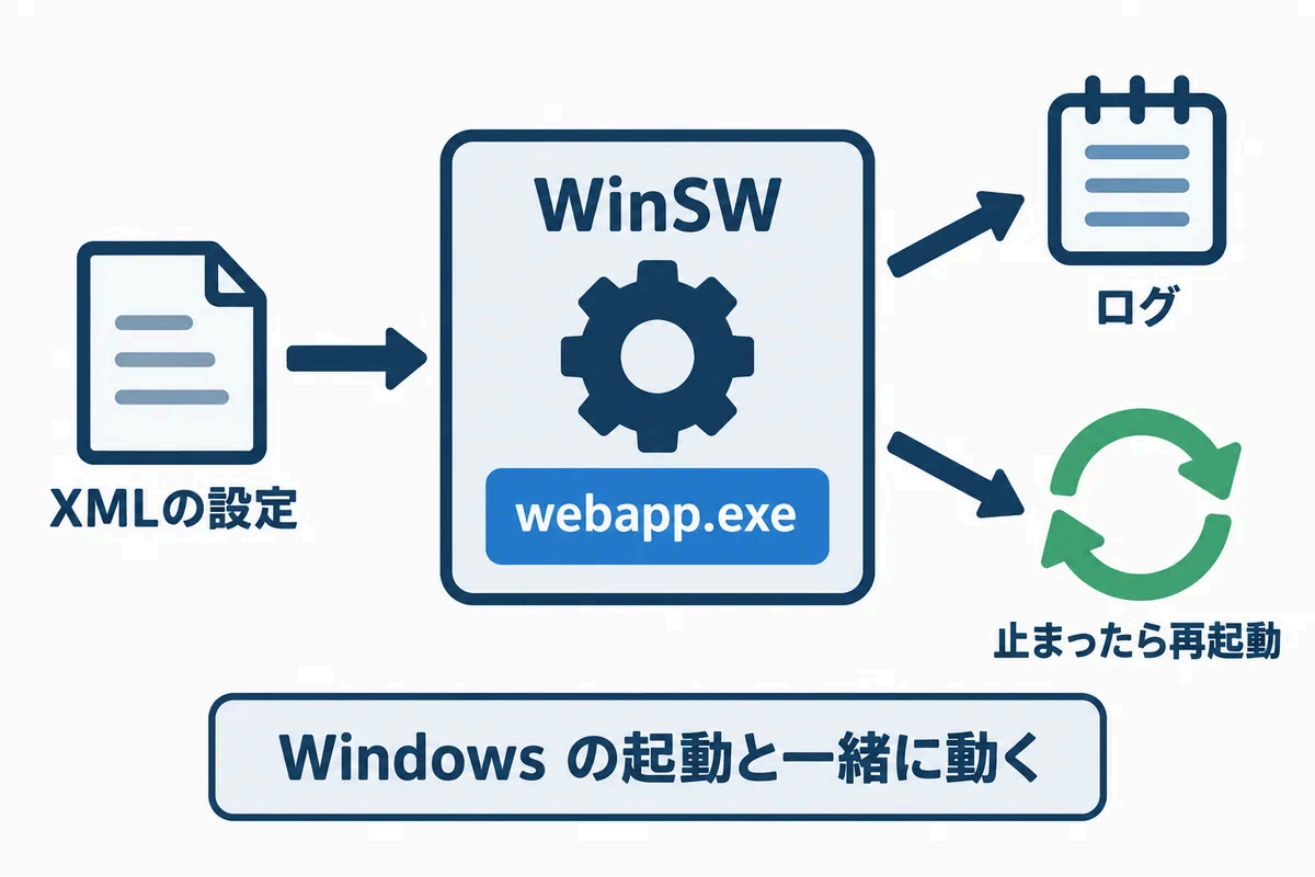



図7-1: WinSW は、ふつうの実行ファイルを包んで Windows サービスにする

Windows サービスは、OS と決まった言葉でやり取りできるプログラムでないとなれません。ふつうの実行ファイルを、そのまま `sc.exe create` でサービスに登録すると、起動のときに「エラー 1053：サービスが時間内に応答しませんでした」で止まります。

**WinSW** は、ふつうの実行ファイルを包んで、サービスにしてくれる道具です。WinSW がサービスとして OS とやり取りし、中で `webapp.exe` を起動し、止まったら起動し直し、画面への出力をログのファイルに書きます。

> **用語:** WinSW（ウィンエスダブリュー。Windows Service Wrapper：ふつうの実行ファイルを、Windows サービスとして動かすための包み紙の道具）

> **用語:** sc.exe（エスシー：Windows サービスを登録・設定する、Windows に付いているコマンド）

## 7.2 WinSW を持ち込む

1. 開発用のパソコンで、WinSW の GitHub のリリースのページ（`https://github.com/winsw/winsw/releases`）から、**v2.12.0** の `WinSW-x64.exe` をダウンロードします。
2. サーバーの `D:\webapp\installers\` にコピーし、ハッシュ値を確かめます。
3. 本番と検証のそれぞれに、**名前を変えて**コピーします。WinSW は、「自分と同じ名前の `.xml` 」を設定ファイルとして読む決まりです。

```
Copy-Item D:\webapp\installers\WinSW-x64.exe D:\webapp\prod\service\webapp-prod-service.exe
Copy-Item D:\webapp\installers\WinSW-x64.exe D:\webapp\stg\service\webapp-stg-service.exe
```

> **用語:** GitHub のリリース（ソフトの作者が、できた版を配っている GitHub のページ）

## 7.3 アプリを置き、設定ファイル（XML）を書く

まず、アプリの zip を、本番の `app\` に展開します。

```
Expand-Archive D:\webapp\releases\webapp-1.0.0.zip -DestinationPath D:\webapp\prod\app
Get-ChildItem D:\webapp\prod\app
```

**確かめること:** `webapp.exe` と `assets` フォルダが、`D:\webapp\prod\app` の**すぐ下**に並んでいる（`app\webapp-1.0.0\webapp.exe` のように1段深くなっていたら、置き直す）。

次に、WinSW の設定ファイル `D:\webapp\prod\service\webapp-prod-service.xml` を、メモ帳で作ります。

```
<service>
  <!-- サービスの名前（コマンドで使う）と、画面に出る名前・説明 -->
  <id>webapp-prod</id>
  <name>業務 Web アプリ（本番）</name>
  <description>業務 Web アプリの本番。127.0.0.1:8080 で待ち受け、IIS から転送を受ける。</description>

  <!-- 動かす実行ファイルと、動かすときの今いるフォルダ -->
  <executable>D:\webapp\prod\app\webapp.exe</executable>
  <workingdirectory>D:\webapp\prod\app</workingdirectory>

  <!-- アプリに渡す設定（環境変数）。HOST と PORT は Topcoat が読む。ほかは設計例 -->
  <env name="HOST" value="127.0.0.1" />
  <env name="PORT" value="8080" />
  <env name="APP_ENV" value="production" />
  <env name="APP_BASE_URL" value="http://webapp.example.local" />
  <env name="DATABASE_URL" value="postgres://webapp_prod_app:（パスワード）@127.0.0.1:5432/webapp_prod" />
  <env name="FILE_DIR" value="D:\webapp\prod\files" />
  <env name="SMTP_HOST" value="example-com.mail.protection.outlook.com" />
  <env name="SMTP_PORT" value="25" />
  <env name="MAIL_FROM" value="noreply@example.com" />
  <env name="RUST_LOG" value="info" />

  <!-- Windows の起動と一緒に動かす。PostgreSQL が動いてから動かす -->
  <startmode>Automatic</startmode>
  <depend>postgresql-x64-18</depend>

  <!-- 止まったら、10 秒後・30 秒後に起動し直す -->
  <onfailure action="restart" delay="10 sec" />
  <onfailure action="restart" delay="30 sec" />

  <!-- 止めるときに待つ時間 -->
  <stoptimeout>15 sec</stoptimeout>

  <!-- 画面への出力を、ログのファイルに書く。10MB ごとに分け、8 個まで残す -->
  <logpath>D:\webapp\prod\logs</logpath>
  <log mode="roll-by-size">
    <sizeThreshold>10240</sizeThreshold>
    <keepFiles>8</keepFiles>
  </log>
</service>
```

| 設定 | 意味 | 本番の値（例） | 検証の値（例） |
|---|---|---|---|
| `HOST`・`PORT` | アプリが待ち受ける住所とポート。Topcoat の `topcoat::start` が読む | `127.0.0.1`・`8080` | `127.0.0.1`・`8081` |
| `APP_ENV` | 本番か検証か（設計例） | `production` | `staging` |
| `APP_BASE_URL` | メールに書く URL の頭（設計例。承認 URL などを作るのに使う） | `http://webapp.example.local` | `http://webapp-stg.example.local` |
| `DATABASE_URL` | データベースの接続文字列（00 基礎知識 7.2） | `webapp_prod` へ | `webapp_stg` へ |
| `FILE_DIR` | 添付ファイルの保存フォルダ（設計例） | `D:\webapp\prod\files` | `D:\webapp\stg\files` |
| `SMTP_HOST`・`SMTP_PORT`・`MAIL_FROM` | メールの渡し先と送り主（10 章。設計例） | Microsoft 365 の MX エンドポイント・25・送信専用のアドレス | 同じ |
| `MAIL_REDIRECT_TO` | （検証だけ）全部のメールを、この宛先に送る（設計例） | 付けない | IT 担当者のアドレスなど |
| `RUST_LOG` | ログの細かさ | `info` | `info` または `debug` |

- Topcoat の `topcoat::start` は、`HOST` と `PORT` の環境変数を読みます（何も無いと `127.0.0.1:3000` ）。**`HOST` は必ず `127.0.0.1`** にします。`0.0.0.0` にすると、IIS を通らずに、外から直接アプリにつながってしまいます。
- `HOST`・`PORT` 以外の名前は、**アプリの作り方で決まる設計例**です。アプリの作者と合わせます。
- 検証の XML は、`id`・`name`・ポート・データベース・フォルダ・`APP_BASE_URL` を検証のものに変え、`MAIL_REDIRECT_TO` を足します。
- **XML ファイルには、データベースのパスワードが書かれます。** 7.4 で、読める人を絞ります。

> **用語:** XML（エックスエムエル： `<名前>中身</名前>` の形で、設定などを書くファイルの形式。`<!-- -->` の中はコメント）

> **用語:** 接続文字列（アプリがデータベースにつなぐための情報を、1行にまとめたもの。`postgres://ユーザー:パスワード@住所:ポート/データベース名` 。パスワードに `@` や `:` などの記号を使ったときは、`%40` のような形に書きかえる必要があるので、パスワードは英字と数字を長くした物にすると間違いが少ない）

## 7.4 サービスを登録し、アカウントと許可を設定する

```
# 1. サービスを登録する（XML の id の名前で登録される）
D:\webapp\prod\service\webapp-prod-service.exe install

# 2. 動かすアカウントを、仮想アカウント NT SERVICE\webapp-prod にする
sc.exe config webapp-prod obj= "NT SERVICE\webapp-prod"

# 3. アカウントを確かめる
Get-CimInstance Win32_Service -Filter "Name='webapp-prod'" | Format-List Name, StartName, StartMode, State
```

**確かめること:** `StartName` が `NT SERVICE\webapp-prod`、`StartMode` が `Auto` になっている。

- `sc.exe` は、`obj=` のように、`=` **の後ろに空白を1つ**入れる決まりです。空白が無いとエラーになります。

次に、仮想アカウントに、フォルダの許可を付けます（4.3 の表のとおり）。

```
# 本番のフォルダ全体：読む・実行する（中にも引き継ぐ）
icacls D:\webapp\prod /grant "NT SERVICE\webapp-prod:(OI)(CI)RX"
# 添付ファイルとログ：書ける
icacls D:\webapp\prod\files /grant "NT SERVICE\webapp-prod:(OI)(CI)M"
icacls D:\webapp\prod\logs  /grant "NT SERVICE\webapp-prod:(OI)(CI)M"

# パスワードが書かれた XML：管理者・SYSTEM・本番のアプリだけが読める（ほかの引き継ぎを切る）
icacls D:\webapp\prod\service\webapp-prod-service.xml /inheritance:r /grant:r "*S-1-5-32-544:F" "*S-1-5-18:F" "NT SERVICE\webapp-prod:R"
```

- 許可を `D:\webapp\prod` に付けておくと、更新のときに `app\` を入れ替えても、新しい `app\` に許可が引き継がれます。
- 検証は、`prod` を `stg` に変えて、同じことをします。本番のアカウントには検証のフォルダの許可を付けず、検証のアカウントには本番のフォルダの許可を付けません。

## 7.5 起動して、サーバーの中で確かめる

```
Start-Service webapp-prod
Get-Service webapp-prod

# アプリが 127.0.0.1:8080 で待ち受けているか
Get-NetTCPConnection -State Listen -LocalPort 8080 | Format-Table LocalAddress, LocalPort, OwningProcess

# アプリの「元気ですか」の入口に聞く（設計例：GET /health）
Invoke-WebRequest http://127.0.0.1:8080/health -UseBasicParsing | Select-Object StatusCode, Content
```

**確かめること:**

- サービスが `Running` になっている。
- 8080 番の `LocalAddress` が `127.0.0.1` になっている。
- `/health` の `StatusCode` が `200` になっている。

**起動しないとき:** `D:\webapp\prod\logs\` の中のログを見ます。

| ファイル | 中身 |
| --- | --- |
| `webapp-prod-service.wrapper.log` | WinSW 自身の記録（起動した・止まった・起動し直した） |
| `webapp-prod-service.out.log` | アプリが画面に出した文（ふつうのログ） |
| `webapp-prod-service.err.log` | アプリが画面に出したエラーの文 |

よくある原因は 15 章の表にまとめています。

> **用語**: ヘルスチェック（health check：アプリに「動いていますか」と聞く、決まった入口（例： `/health` ）。動いていれば 200 を返す。監視やデプロイの確かめに使う）

## 7.6 テーブルを作る（初回のマイグレーション）

> **設計例**: アプリは、起動したときに、まだ当てていないマイグレーションを当てる。

この作りなら、7.5 で最初に起動したときに、テーブルができています。確かめます。

```
psql -h 127.0.0.1 -U webapp_prod_app -d webapp_prod -c "\dt"
```

**確かめること**: テーブルの一覧（依頼書・図面・社員データ・承認の記録など。名前はアプリの作りによる）と、sqlx がマイグレーションの記録に使う `_sqlx_migrations` が出る。

## 7.7 初期データを登録する

**最初の管理者を決める（サーバー上のコマンド）**

画面から権限を設定するには、権限を設定できる人（管理者）が要ります。最初の1人は、サーバーの上でコマンドを使って決めます。2人目からは、画面で設定します。

> **設計例**: `webapp.exe` に、`grant-role` という管理用のコマンドを用意する。

```
# この窓の中だけ、データベースの接続先を教える（サービスの XML と同じ値）
$env:DATABASE_URL = "postgres://webapp_prod_app:（パスワード）@127.0.0.1:5432/webapp_prod"
D:\webapp\prod\app\webapp.exe grant-role --employee-no 012345 --role it-admin
# 終わったら、窓の中の値を消す
Remove-Item Env:DATABASE_URL
```

ただし、社員データ（従業員番号）が無いと、権限を付ける相手がいません。アプリの作りに合わせて、順番を決めます（例：コマンドで CSV を先に取り込む、または、コマンドで最初の管理者を社員データごと作る）。

**社員データの CSV を取り込む（画面）**

最初の管理者でログインし、社員データの取り込みの画面から、社員一覧の CSV を取り込みます。CSV の列の並びは、アプリの取り込みの決まりに合わせます。

- **先に検証環境で取り込んで、結果を確かめてから**、本番に取り込みます。
- 社員の個人情報なので、CSV のファイルは、取り込んだあとサーバーやパソコンに残さず、会社の決まりに従って扱います。

> **用語:** CSV（シーエスブイ：表のデータを、カンマで区切った文字で書いたファイル。Excel で開ける）

## 8. 受付（IIS）

社員のブラウザからのお願いを最初に受ける、受付（IIS）を用意します。今回は HTTPS を使えないので、**HTTP（80 番）**で受けます。HTTPS にするときの手順は、参考資料「HTTPS にする場合」にあります。

> **用語:** IIS（Windows Server に付いている Web サーバー。00 基礎知識 5.2）

> **用語:** リバースプロキシ（外からのお願いを受けて、奥のアプリに取り次ぐ係。00 基礎知識 5.2）

### 8.1 IIS を入れる

```
Install-WindowsFeature -Name Web-Server -IncludeManagementTools
Get-WindowsFeature Web-Server, Web-Mgmt-Console | Format-Table Name, InstallState
```

**確かめること:** `Web-Server` と `Web-Mgmt-Console` が `Installed` になっている。

- IIS を入れると、`Default Web Site`（見本のサイト）が 80 番で動き始めます。今回は使わないので、止めます（8.4）。

> **用語:** 役割と機能（Windows Server に、あとから足せる機能のまとまり。IIS は「Web サーバー（IIS）」の役割。`Install-WindowsFeature` で足す）

### 8.2 URL Rewrite と ARR を入れる

IIS は、そのままではリバースプロキシになれません。Microsoft が配っている、2つの追加の部品を入れます。**入れる順番は、URL Rewrite が先、ARR が後**です。

| 部品 | 役目 | 入手先（開発用のパソコンでダウンロードして持ち込む） |
|---|---|---|
| URL Rewrite 2.1 | 来た URL を見て、「どこへ渡すか」の決まり（ルール）を当てる | Microsoft の IIS のページの「URL Rewrite」から、x64 の英語版（`rewrite_amd64_en-US.msi`） |
| Application Request Routing 3.0（ARR） | ルールで決めた行き先（アプリ）へ、実際にお願いを取り次ぐ | Microsoft の IIS のページの「Application Request Routing」から、x64 版（`requestRouter_amd64.msi`） |

1. 2つの `.msi` を `D:\webapp\installers\` にコピーし、ハッシュ値を確かめます。
2. `rewrite_amd64_en-US.msi` をダブルクリックして入れます。
3. `requestRouter_amd64.msi` をダブルクリックして入れます。
4. IIS マネージャー（`inetmgr`）を開き直し、サーバーの名前を選んだ画面に「URL Rewrite」と「Application Request Routing Cache」の絵が出ていることを確かめます。

> **用語:** URL Rewrite（ユーアールエル リライト：来た URL を、決まりに従って書きかえたり、別の所へ渡したりする IIS の部品）

> **用語:** ARR（エーアールアール。Application Request Routing：IIS が受けたお願いを、別のサーバーやアプリに取り次ぐための IIS の部品）

> **用語:** msi（エムエスアイ：Windows のインストーラーのファイルの形式の1つ）

## 8.3 ARR の取り次ぎを有効にし、Host をそのまま渡す

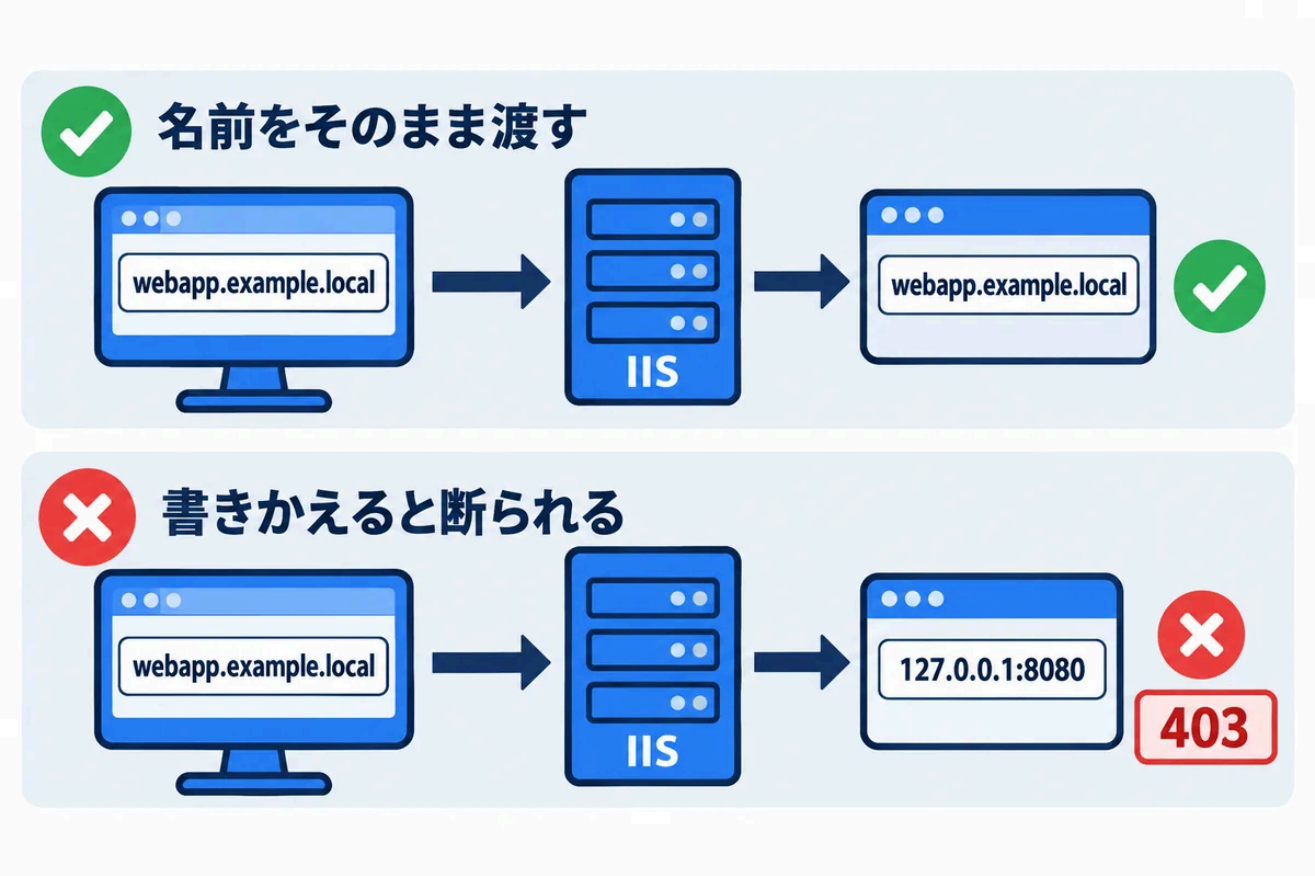



図8-1: IIS が名前（Host）を書きかえると、アプリに断られる

ARR は、入れただけでは取り次ぎをしません。サーバー全体の設定で、有効にします。あわせて、**Host ヘッダー（どの名前で来たか）をそのままアプリに渡す**設定にします。**HTTP で動かすときは、この設定が必須**です（下の説明）。

```
$appcmd = "$env:windir\System32\inetsrv\appcmd.exe"

# 取り次ぎ（プロキシ）を有効にする
& $appcmd set config -section:system.webServer/proxy /enabled:"True" /commit:apphost

# Host ヘッダーをそのまま渡す（既定では、取り次ぎ先の名前（127.0.0.1:8080）に書きかえられる）
& $appcmd set config -section:system.webServer/proxy /preserveHostHeader:"True" /commit:apphost

# 結果を見る
& $appcmd list config -section:system.webServer/proxy
```

**確かめること**: 結果に `enabled="true"` と `preserveHostHeader="true"` が出ている。

なぜ Host をそのまま渡すのか:

- Topcoat には、ほかのサイトのページから勝手にフォームを送らせる攻撃（CSRF）を止める仕組み（OriginPolicy）があります。Topcoat 0.11 のソースでは、まずブラウザが付ける `Sec-Fetch-Site` を見て、無いときは `Origin`（どのサイトのページから送ったか）と `Host` を比べます。

- ブラウザは、`Sec-Fetch-Site` を HTTPS のページ（と、自分のパソコンの中の localhost）にしか付けません。**HTTP では、いつも `Origin` と `Host` の比べ合わせになります。**
- Host が `127.0.0.1:8080` に書きかえられると、`Origin: http://webapp.example.local` と合わないので、自分のサイトからの送信（ログイン・保存・承認など）が全部 `403 Forbidden` になります。
- 実際に、Topcoat 0.11 のアプリ（Rust の教材の RustNote）で、HTTP のまま試しました（2026-11-01）。`Origin` と `Host` が違う POST は `403`、そろえた POST は通りました（`303`）。
- 承認 URL のように、アプリが自分の URL を作るときにも、正しい名前が要ります（このアプリでは `APP_BASE_URL` を使う設計例）。

> **用語:** ヘッダー（リクエストやレスポンスに付いている、追加の情報の行。`Host`（どの名前宛てか）、`Cookie`、`Origin` など）

> **用語:** CSRF（シーサーフ：ほかのサイトのページから、ログイン中の人のブラウザに、勝手に操作を送らせる攻撃）

> **用語:** appcmd（アップコマンド：IIS の設定を、コマンドで変える道具。`/commit:apphost` は「サーバー全体の設定ファイルに書く」の意味）

## 8.4 サイトを作る

本番と検証の2つのサイトを、**同じ 80 番で、名前で分けて**作ります（00 基礎知識 4.2 の Host ヘッダー）。

```
Import-Module IISAdministration

# 見本のサイトを止めて、自動で動かないようにする
Stop-IISSite -Name "Default Web Site" -Confirm:$false
Import-Module WebAdministration
Set-ItemProperty "IIS:\Sites\Default Web Site" -Name serverAutoStart -Value $false

# 本番のサイト（http://webapp.example.local/）
New-IISSite -Name "webapp-prod" -PhysicalPath "D:\webapp\prod\iis" -BindingInformation "*:80:webapp.example.local"

# 検証のサイト（http://webapp-stg.example.local/）
New-IISSite -Name "webapp-stg" -PhysicalPath "D:\webapp\stg\iis" -BindingInformation "*:80:webapp-stg.example.local"

Get-IISSite
```

**確かめること:** `webapp-prod` と `webapp-stg` が `Started` で、`Default Web Site` が `Stopped` になっている。

- 見本のサイトを止めておくと、IP アドレス（`http://10.20.30.40/`）や、登録していない名前で来たお願いには、何も出しません。

**IIS がサイトのフォルダを読めるようにする:** `D:\webapp` は 4.2 で管理者と SYSTEM だけにしたので、IIS のアプリケーションプール（IIS のサイトを動かすプロセス）が `web.config` を読めるように、`iis\` のフォルダにだけ許可を付けます。

```
# *S-1-5-32-568 は IIS_IUSRS（IIS のアプリケーション プールが入るグループ）
icacls D:\webapp\prod\iis /grant "*S-1-5-32-568:(OI)(CI)RX"
icacls D:\webapp\stg\iis  /grant "*S-1-5-32-568:(OI)(CI)RX"
```

> **用語:** バインド（Binding：サイトと「どの IP・ポート・名前で受けるか」を結び付ける設定。
> `*:80:webapp.example.local` は「どの IP でも、80 番で、この名前で来たら、このサイト」）

> **用語:** アプリケーション プール（IIS の中で、サイトを動かすプロセスのまとまり）

## 8.5 取り次ぎのルール（web.config）を書く

`D:\webapp\prod\iis\web.config` を、メモ帳で作ります。

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
  <system.webServer>
    <rewrite>
      <rules>
        <!-- 来たお願いを全部、本番のアプリ（127.0.0.1:8080）へ取り次ぐ -->
        <rule name="to-webapp-prod" stopProcessing="true">
          <match url="(.*)" />
          <action type="Rewrite" url="http://127.0.0.1:8080/{R:1}" />
        </rule>
      </rules>
    </rewrite>
    <security>
      <requestFiltering>
        <!-- 1 回に送れる大きさの上限（バイト）。100MB。理由は下の表 -->
        <requestLimits maxAllowedContentLength="104857600" />
      </requestFiltering>
    </security>
  </system.webServer>
</configuration>
```

検証の `D:\webapp\stg\iis\web.config` は、ルールの名前を `to-webapp-stg` に、行き先を `http://127.0.0.1:8081/{R:1}` にします。

| 部分 | 意味 |
| --- | --- |
| `<match url="(.*)" />` | どの URL でも当てはまる。`(.*)` の中身が `{R:1}` に入る |
| `<action type="Rewrite" url="http://127.0.0.1:8080/{R:1}" />` | 同じ道（パス）のまま、アプリへ取り次ぐ。`?` の後ろ（クエリ文字列）も、そのまま付く |
| `maxAllowedContentLength` | IIS が受け付ける、1 回の送信の大きさの上限。**既定は約 28.6MB**（30,000,000 バイト） |

送信の大きさの上限は、例のアプリの「添付ファイルは 1 ファイル 10MB まで、図面は 1 枚 5MB まで・1 件に 10 枚まで」から決めます。

- 画面の作りによっては、1 回の送信に、何ファイルかがまとめて入ります。既定の約 28.6MB では足りないことがあるので、例では 100MB にしています。
- **1 ファイルごとの上限（10MB・5MB）は、アプリが確かめます。** IIS の上限は「それより大きい物は、アプリに届く前に断る」ための、外側の柵です。
- 上限を超えると、IIS は `404.13` を返します（アプリのログには何も出ません）。
- **アプリの側にも上限があります。** Topcoat 0.11 は、決まりでは 2MB より大きい送信を `413` で断ります。添付ファイルや図面を受け取る画面だけ、`BodyLimit` で上限を上げます（webapp_learning の 08 後編の 5.3。練習アプリでは `/requests` の下だけ 60MB）。IIS の上限と、アプリの上限の、**小さい方**が効きます。

> **用語:** web.config（ウェブコンフィグ：IIS のサイトごとの設定を書く XML のファイル。サイトのフォルダに置く）

> **用語:** パス（URL の、名前の後ろの `/requests/123` の部分）・クエリ文字列（URL の `?` の後ろの、`?page=2` のような部分）

## 8.6 社員のパソコンから開く

DNS に名前が登録されたら（情報システム部門の作業）、**自分のパソコン**で確かめます。

```
# 自分のパソコンの PowerShell で
Resolve-DnsName webapp-stg.example.local
Test-NetConnection webapp-stg.example.local -Port 80
```

**確かめること:**

- `Resolve-DnsName` で、サーバーの IP アドレス（例：`10.20.30.40`）が出る。
- `Test-NetConnection` で、`TcpTestSucceeded : True` が出る。
- ブラウザ（Edge・Chrome）で `http://webapp-stg.example.local/health` を開くと、アプリの返事が出る。アドレス欄に「保護されていない通信」などと出るのは、HTTP なので正しい表示です。

- ログインして、ほかの画面へ移っても、ログインが続く（6.4 のクッキーの設定が効いている）。

## 9. ファイアウォール

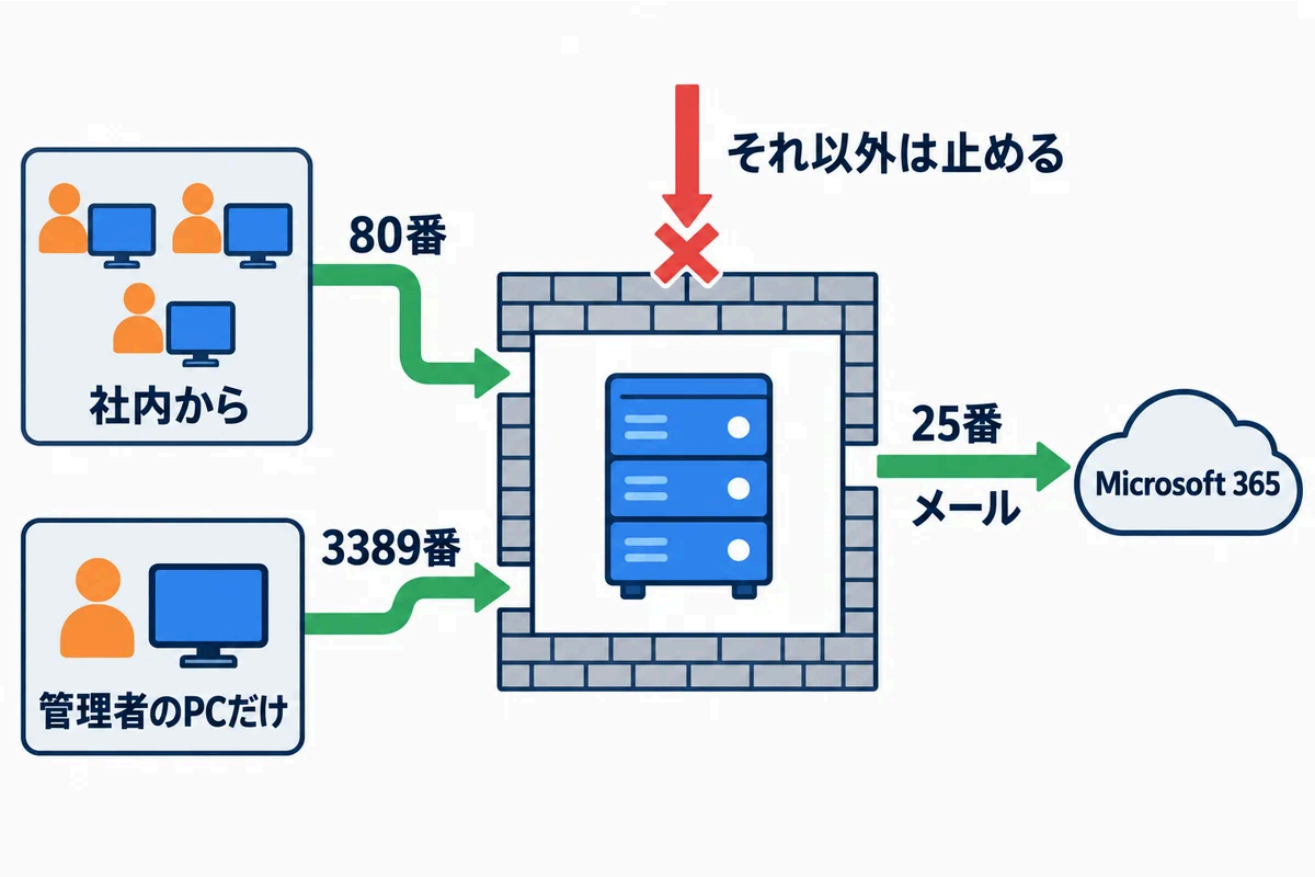



図9-1: サーバーの関所で通す通信は 3 つだけ

社内からの HTTP（80 番）だけを受け付け、外向きは Microsoft 365 へのメール送信（25 番）だけを通すようにします。関所は2か所あります（00 基礎知識 4.3）。

| 関所 | 誰が設定するか | この章ですること |
| --- | --- | --- |
| ネットワークの途中の関所（会社の機器） | 情報システム部門 | 依頼して、通ったかを確かめる |
| サーバーの中の関所（Windows Defender ファイアウォール） | 自分（会社の決まりで、情報システム部門が配っていることもある） | 受信の 80 を社内からだけにする。ほかの入口を閉じる |

## 9.1 今の状態を見る

```
# どのプロファイル（ドメイン・プライベート・パブリック）で動いているか
Get-NetConnectionProfile | Format-Table Name, NetworkCategory

# 80 番・443 番の受信の決まりを探す（IIS を入れると、自動で作られることがある）
Get-NetFirewallPortFilter | Where-Object { $_.LocalPort -in 80, 443 } | Get-NetFirewallRule |
  Format-Table DisplayName, Enabled, Direction, Action, Profile
```

> **用語:** プロファイル（Windows ファイアウォールの、場所ごとの決まりのまとまり。会社のドメインにつながっているときは「ドメイン」（DomainAuthenticated））

## 9.2 受信の決まりを作る

```
# 社内からの 80 番を通す（社内の IP の範囲は、情報システム部門に聞いて入れる。例：10.0.0.0/8）
New-NetFirewallRule -DisplayName "webapp HTTP 80（社内から）" -Direction Inbound -Protocol TCP `
  -LocalPort 80 -RemoteAddress 10.0.0.0/8 -Action Allow -Profile Domain

# IIS が自動で作った、誰からでも 80・443 を通す決まりは、止める
Get-NetFirewallPortFilter | Where-Object { $_.LocalPort -in 80, 443 } | Get-NetFirewallRule |
  Where-Object { $_.DisplayName -notlike "webapp*" } | Disable-NetFirewallRule
```

- リモート デスクトップ（3389 番）の決まりは、情報システム部門が決めていることが多いので、勝手に変えません。**変えると、自分がサーバーに入れなくなることがあります。**
- 外向き（送信）の決まりは、サーバーの中では止めません。サーバーの中で外向きを止めると、ドメインのサーバーとのやり取りや Windows Update まで止まってしまうためです。外向きの「Microsoft 365 への 25 番だけ」は、**ネットワークの途中の関所**で、情報システム部門が守ります。

## 9.3 外から見えてはいけない物が見えていないか

```
# このサーバーで待ち受けているポートと住所の一覧
Get-NetTCPConnection -State Listen | Sort-Object LocalPort |
  Format-Table LocalAddress, LocalPort, OwningProcess -AutoSize
```

**確かめること:**

- `5432`（PostgreSQL）・`8080`・`8081`（アプリ）の `LocalAddress` が、`127.0.0.1` か `::1` だけ。
- `80` は `0.0.0.0` または `::`（全部の住所）で待ち受けている（IIS。これは正しい）。
- 自分のパソコンから `Test-NetConnection 10.20.30.40 -Port 8080` を打って、`TcpTestSucceeded : False` になる（外からアプリに直接つながらない）。

> **用語:** 0.0.0.0（「このコンピューターの全部の住所」を表す書き方。ここで待ち受けると、外からもつながる）

## 10. メール（Microsoft 365 の SMTP リレー）

Microsoft 365 にコネクタと送信元アドレスを作ってもらい、アプリからテストのメールを送れるようにします。

> **用語:** SMTP リレー（Microsoft 365 にメールの配達を任せる方式。00 基礎知識 8.2）

### 10.1 仕組みと、頼むこと

Microsoft の公式のドキュメントによると、SMTP リレーは次の条件で使えます（2026-10 時点）。

- アプリは、会社のドメインの **MX エンドポイント**（例：`example-com.mail.protection.outlook.com`）の **25 番**へ、SMTP でメールを渡す。
- Microsoft 365 の管理画面（Exchange 管理センター）に、**コネクタ**（送り元：組織のメールサーバー → 送り先：Office 365）を作る。送り主は、**会社のインターネットの出口の、固定の公開 IP アドレス**で見分ける（ほかの会社と共有していないもの）。
- 送り主のアドレスは、会社が使っているドメインのアドレスなら、メールボックスが無くてもよい（例：`noreply@example.com`）。
- 迷惑メールと判定されないように、ドメインの SPF に、出口の IP アドレスを書き足す。

これらは全部、**情報システム部門の作業**です。頼む内容は、07 付録の依頼シートにまとめます。ID とパスワードで送る方式（SMTP 認証の基本認証）は、Microsoft が順に使えなくしていくと案内しているので、使いません。

> **用語:** Exchange 管理センター（Microsoft 365 のメールの設定をする管理画面）

> **用語:** 公開 IP アドレス（インターネットの上で使われる住所。会社の中の住所（10.... など）とは別。会社の出口の機器が、中の住所を公開 IP アドレスに置きかえて外に出す）

### 10.2 メールを渡す先へつながるか確かめる

```
# 会社のドメインの MX（メールの受付）を調べる。example.com は会社のドメインに置きかえる
Resolve-DnsName example.com -Type MX

# 調べた MX エンドポイントの 25 番に、サーバーからつながるか
Test-NetConnection example-com.mail.protection.outlook.com -Port 25
```

**確かめること:**

- `Resolve-DnsName` の `NameExchange` に、`…mail.protection.outlook.com` が出る。
- `Test-NetConnection` で `TcpTestSucceeded : True` が出る。`False` なら、ネットワークの途中の関所で 25 番が通っていない（情報システム部門に確かめる）。

## 10.3 テストのメールを送る

**まず、アプリを使わずに送ってみます。**うまくいかないとき、「アプリの不具合か、メールの道の問題か」を分けられるからです。PowerShell 5.1 の `Send-MailMessage` は、古い道具として「使わないように」と案内されていますが、つながるかの試しには使えます。

```
Send-MailMessage -SmtpServer example-com.mail.protection.outlook.com -Port 25 -UseSsl `
  -From "noreply@example.com" -To "自分の会社のアドレス" `
  -Subject "業務 Web アプリ 送信テスト" -Body "サーバーからの送信テストです。" -Encoding UTF8
```

**確かめること:** 自分の Outlook に、迷惑メールのフォルダではなく、受信トレイに届く。

うまくいったら、次に**アプリから**送ります（設計例：管理者の設定の画面に「テストメールを送る」ボタンを用意する。または管理用のコマンド）。

**よくある失敗:**

| 出る文（例） | 考えられる原因 |
|---|---|
| `TcpTestSucceeded : False`、または「接続できませんでした」 | 25 番が、ネットワークの途中の関所で止められている |
| `550 5.7.64 TenantAttribution; Relay Access Denied` | コネクタが無い、またはコネクタの IP アドレスと、会社の出口の IP アドレスが違う |
| 届くが、迷惑メールのフォルダに入る | SPF に、出口の IP アドレスが書かれていない |

## 10.4 検証のメールを、社員に送らない

検証環境で、本物の社員に「承認のお願い」が届くと、混乱します。

> **設計例:** `APP_ENV=staging` のときは、`MAIL_REDIRECT_TO` の宛先だけに送り、件名の頭に `【検証】` を付ける。本来の宛先は本文の先頭に書く。

# 11. 毎晩のバックアップ

## 11.1 何を、どこに、どれだけ残すか

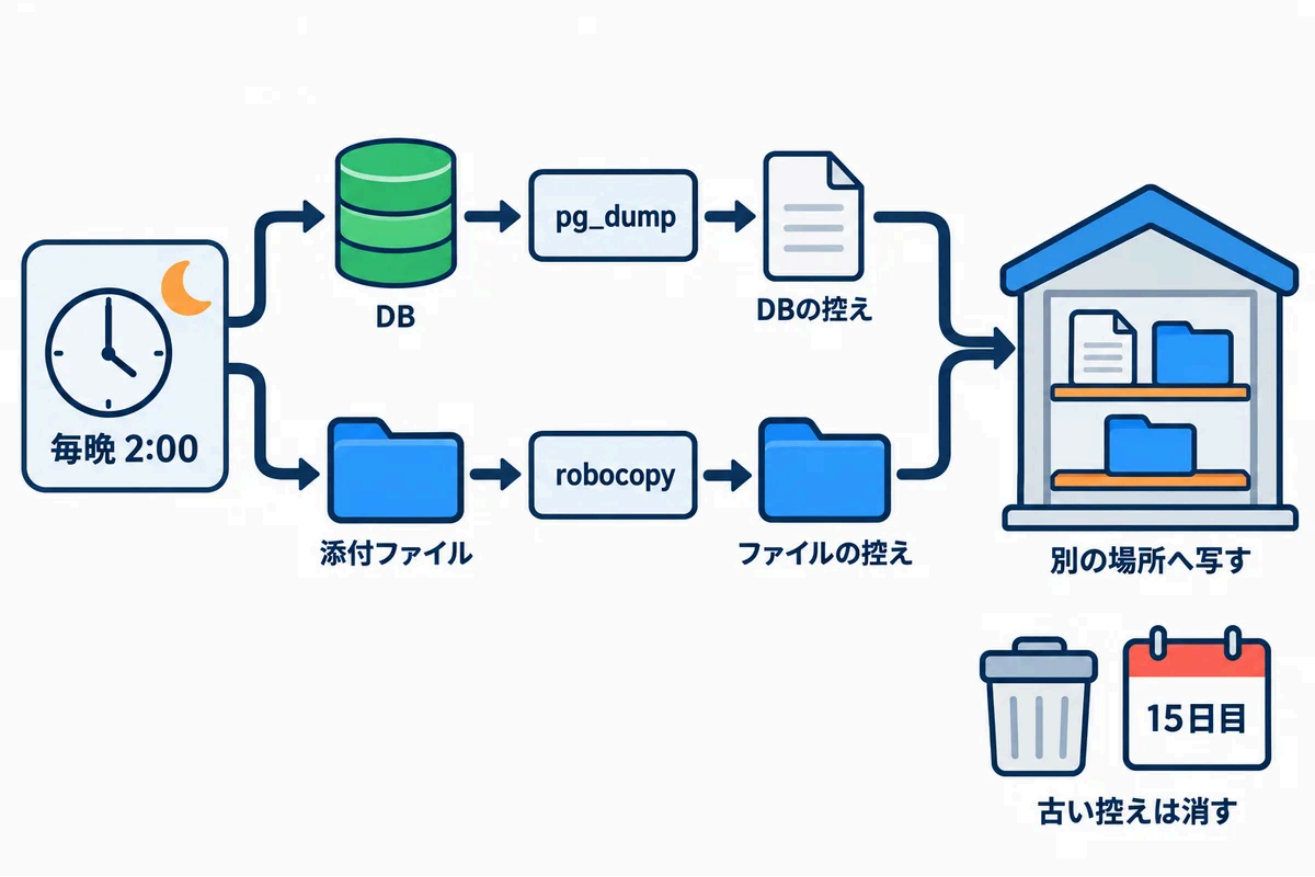



図11-1: 毎晩のバックアップの流れ

| 何を | どうやって | どこに | どれだけ |
|---|---|---|---|
| 本番のデータベース | `pg_dump` で 1 つのファイルに書き出す | `D:\webapp\backup\db\` と、別の場所 | 14 日分 |
| 本番の添付ファイル | `robocopy` で写す（消された物は消さずに残す） | `D:\webapp\backup\files\current\` と、別の場所 | 全部（下の説明） |
| バックアップの記録 | 実行の記録を書く | `D:\webapp\backup\logs\` | 14 日分 |

**添付ファイルを「毎日まるごと 14 回分」写さない理由:** 添付ファイルは 5 年分たまるので、毎日まるごと写すと、14 倍の場所が要ります。

> **設計例:** アプリは、添付ファイルを上書きせず、いつも新しい名前で保存する。消すときも、データベースの印を変えるだけで、ファイルはすぐには消さない。

この作りなら、「写し先から消さずに、新しいファイルを足していく」だけで、どの日のデータベースに戻しても、そのとき使っていた添付ファイルが全部そろいます。データベースは 14 日分の控えがあるので、「14 日のうちのどの日にも戻せる」状態になります。

**別の場所にも写す理由:** 同じサーバーの中だけに控えを置くと、サーバーやディスクが壊れたときに、本物と控えを一緒に失います。仮想マシンのスナップショットも、バックアップの代わりにはなりません（00 基礎知識 9.2）。

> **用語:** pg_dump（ピージーダンプ：PostgreSQL のデータベースの中身を、1 つのファイルに書き出す道具。`-Fc` は、小さくまとめた PostgreSQL 専用の形（あとで `pg_restore` で戻す））

> **用語:** robocopy（ロボコピー：Windows に付いている、フォルダを丸ごと、途中で切れても続けて写せる道具。終了コードが 8 より小さければ成功）

## 11.2 パスワードのファイルを作る

バックアップは、人がいない夜中に動くので、データベースのパスワードを打てません。PostgreSQL の決まりのパスワードのファイル（pgpass）に書いておき、そのファイルを読めるのを管理者と SYSTEM だけにします。

`D:\webapp\secrets\pgpass.conf` を、メモ帳で作ります（1 行。`住所:ポート:データベース:ユーザー:パスワード`）。

```
127.0.0.1:5432:webapp_prod:webapp_prod_app:（本番のアプリ用のパスワード）
```

`D:\webapp\secrets` は、4.2 で管理者と SYSTEM だけになっています。念のため確かめます。

```
icacls D:\webapp\secrets\pgpass.conf
```

> **用語:** pgpass（ピージーパス：PostgreSQL の道具が、パスワードを打たずにつなぐために読むファイル。環境変数 `PGPASSFILE` で場所を教える）

## 11.3 バックアップのスクリプト

`D:\webapp\tools\backup.ps1` を、メモ帳で作ります。

**必ず「UTF-8 (BOM 付き)」で保存します。**メモ帳の「名前を付けて保存」の画面の下の「エンコード」で、「UTF-8 (BOM 付き)」を選びます。Windows PowerShell 5.1 は、BOM（ファイルの先頭の印）の無い UTF-8 のファイルを Shift_JIS として読むため、日本語のコメントの所で行の区切りが食われて、**スクリプトが丸ごと動きません**（このスクリプトを BOM 無しで保存して試すと、構文エラーが 2 つ出て、終了コード 1 で止まりました）。保存したら、次で確かめます。

```powershell
# 先頭の 3 つの数が 239 187 191 なら BOM 付き
```

(Get-Content D:¥webapp¥tools¥backup.ps1 -Encoding Byte -TotalCount 3) -join ' ' ```

```powershell
# D:\webapp\tools\backup.ps1
# 毎晩、本番のデータベースと添付ファイルの控えを取り、別の場所へも写し、14 日より古い控えを消す。
# 失敗したら、終了コード 1 で終わる（タスク スケジューラの「前回の実行結果」で分かる）。

$ErrorActionPreference = "Stop"
$stamp   = Get-Date -Format "yyyyMMdd_HHmm"
$keepDay = 14
$pgBin   = "C:\Program Files\PostgreSQL\18\bin"
$root    = "D:\webapp\backup"
$share   = "\\fileserver\backup\webapp"   # 別の場所（例）。情報システム部門に用意してもらう
$env:PGPASSFILE = "D:\webapp\secrets\pgpass.conf"

Start-Transcript -Path (Join-Path $root "logs\backup_$stamp.log") | Out-Null
try {
    # 1. データベース（本番）を 1 つのファイルに書き出す
    $dump = Join-Path $root "db\webapp_prod_$stamp.dump"
    & "$pgBin\pg_dump.exe" -h 127.0.0.1 -U webapp_prod_app -d webapp_prod -Fc -f $dump
    if ($LASTEXITCODE -ne 0) { throw "pg_dump が失敗しました（終了コード $LASTEXITCODE）" }

    # 2. 添付ファイル（本番）を写す。/E は中のフォルダも全部。/MIR は使わない（消された物を消さない）
    robocopy "D:\webapp\prod\files" (Join-Path $root "files\current") /E /R:2 /W:5 /NP /NFL /NDL
    if ($LASTEXITCODE -ge 8) { throw "robocopy（添付ファイル）が失敗しました（終了コード $LASTEXITCODE）" }

    # 3. 別の場所へ写す
    robocopy (Join-Path $root "db") (Join-Path $share "db") "webapp_prod_$stamp.dump" /R:2 /W:5 /NP
    if ($LASTEXITCODE -ge 8) { throw "robocopy（別の場所・DB）が失敗しました（終了コード $LASTEXITCODE）" }
    robocopy (Join-Path $root "files\current") (Join-Path $share "files") /E /R:2 /W:5 /NP /NFL /NDL
    if ($LASTEXITCODE -ge 8) { throw "robocopy（別の場所・添付）が失敗しました（終了コード $LASTEXITCODE）" }

    # 4. 14 日より古い控えを消す（このサーバーと、別の場所）
    $limit = (Get-Date).AddDays(-$keepDay)
    foreach ($dir in @((Join-Path $root "db"), (Join-Path $share "db"))) {
        Get-ChildItem $dir -Filter "webapp_prod_*.dump" | Where-Object { $_.LastWriteTime -lt $limit } | Remove-Item
    }
    Get-ChildItem (Join-Path $root "logs") -Filter "backup_*.log" | Where-Object { $_.LastWriteTime -lt $limit } | Remove-Item

    Write-Output "OK: $dump"
    Stop-Transcript | Out-Null
    exit 0
}
catch {
    Write-Output "NG: $_"
    Stop-Transcript | Out-Null
    exit 1
}
```

| 部分 | 意味 |
| --- | --- |
| `$ErrorActionPreference = "Stop"` | PowerShell のコマンドが失敗したら、そこで止めて `catch` へ進む |
| `$LASTEXITCODE` | 直前に動かしたプログラム（`pg_dump`・`robocopy`）の終了コード |
| `Start-Transcript` | この窓に出た文を、全部ファイルに書く（あとで何が起きたかを見る） |
| `robocopy` の `/R:2 /W:5` | 失敗したら 5 秒待って 2 回までやり直す |
| `robocopy` の `/NP /NFL /NDL` | 進み具合・ファイル名・フォルダ名を記録に出さない（記録が大きくなりすぎないように） |

> **用語:** 終了コード（プログラムが終わるときに返す番号。ふつう 0 が成功。robocopy だけは 0〜7 が成功、8 以上が失敗）

## 11.4 タスク スケジューラに登録する

```
$action    = New-ScheduledTaskAction -Execute "powershell.exe" `
               -Argument '-NoProfile -ExecutionPolicy Bypass -File "D:\webapp\tools\backup.ps1"'
$trigger   = New-ScheduledTaskTrigger -Daily -At "02:00"
$principal = New-ScheduledTaskPrincipal -UserId "SYSTEM" -LogonType ServiceAccount -RunLevel Highest
$settings  = New-ScheduledTaskSettingsSet -ExecutionTimeLimit (New-TimeSpan -Hours 2) -StartWhenAvailable

Register-ScheduledTask -TaskPath "\webapp\" -TaskName "webapp-backup" -Action $action -Trigger $trigger `
  -Principal $principal -Settings $settings -Description "業務 Web アプリ 毎晩のバックアップ（本番）"

# 今すぐ 1 回動かして、結果を見る
Start-ScheduledTask -TaskPath "\webapp\" -TaskName "webapp-backup"
Get-ScheduledTaskInfo -TaskPath "\webapp\" -TaskName "webapp-backup" | Format-List LastRunTime, LastTaskResult, NextRunTime
```

**確かめること:**

- `LastTaskResult` が `0`（成功）。動いている途中は `267009` が出るので、少し待って打ち直す。
- `D:\webapp\backup\db\` に `.dump` のファイルができ、別の場所にも同じファイルがある。
- `D:\webapp\backup\logs\` の記録の最後に `OK:` が出ている。
- `SYSTEM` で動かすので、**別の場所（共有フォルダ）には、このサーバーのコンピューターのアカウント**（`ドメイン名\WEBAPP-SV01$` のような名前）で書き込みます。共有フォルダの許可を、情報システム部門に付けてもらいます。
- `-StartWhenAvailable` は、2:00 にサーバーが止まっていたら、動き出したあとで実行する設定です。

> **用語:** タスク スケジューラ（決まった時刻にコマンドを動かす Windows の機能。00 基礎知識 9.3）

> **用語:** 実行ポリシー（ExecutionPolicy：PowerShell のスクリプトを動かしてよいかの決まり。`-ExecutionPolicy Bypass` は、このときだけ決まりを気にせず動かす。会社の決まりで禁止されているときは、情報システム部門と相談する）

### 11.5 戻せることを確かめる（リストアの練習）

**バックアップは、戻せることを確かめて、初めて意味があります。**最初に設定したときと、その後も定期的に（例：3 か月に 1 回）練習します。本番のデータベースは変えずに、**練習用のデータベース**に戻します。

```
$env:Path += ";C:\Program Files\PostgreSQL\18\bin"
$dump = (Get-ChildItem D:\webapp\backup\db\webapp_prod_*.dump | Sort-Object LastWriteTime | Select-Object -Last 1).FullName

# 練習用のデータベースを作り、控えを戻す（スーパーユーザーで）
psql -h 127.0.0.1 -U postgres -c "CREATE DATABASE webapp_restore_test ENCODING 'UTF8' TEMPLATE template0;"
pg_restore -h 127.0.0.1 -U postgres -d webapp_restore_test --no-owner --no-privileges $dump

# 中身を見る（テーブルの名前はアプリの作りによる。例：依頼書のテーブル）
psql -h 127.0.0.1 -U postgres -d webapp_restore_test -c "\dt"

# 練習が終わったら消す
psql -h 127.0.0.1 -U postgres -c "DROP DATABASE webapp_restore_test;"
```

**確かめること:** `pg_restore` がエラー無しで終わり、テーブルの一覧と、件数（`SELECT count(*) FROM テーブル名;`）が本番と同じくらいになっている。

> **用語:** pg_restore（ピージーリストア：`pg_dump -Fc` で書き出したファイルを、データベースに戻す道具）

## 12. 定期処理・ログ・監視

### 12.1 毎朝 8:00 の処理

例のアプリでは、毎朝 8:00 の「期限のフォロー・承認の停滞・催促」の判定とメール送信を、**アプリの中の定期処理**として動かします。サーバーの側でする設定はありませんが、次のことに気を付けます。

- サーバーの時刻とタイムゾーンが正しいこと（3.3・3.4）。
- 8:00 にアプリが止まっていると、その日の処理が動かない。**設計例:** アプリは、起動したときに「今日の 8:00 の処理が済んでいるか」を見て、済んでいなければ実行する。
- Windows Update の再起動の時間を、8:00 の前後に重ねない。
- 検証環境でも同じ処理が動く。検証のメールは決まった宛先だけに送る（10.4）。

## 12.2 ログの場所と、たまりすぎを防ぐ

| ログ | 場所 | たまりすぎを防ぐ方法 |
|---|---|---|
| アプリ（WinSW が受け取った出力） | `D:\webapp\prod\logs\` | WinSW の `roll-by-size`（10MB ごと・8 個まで。7.3） |
| IIS のアクセスログ | `C:\inetpub\logs\LogFiles\W3SVC（番号）\` | 自動では消えない。下のタスクで、90 日より古い物を消す（期間は会社の決まりに合わせる） |
| PostgreSQL | `D:\pgsql\18\data\log\` | 既定で日ごとに分かれる。たまり具合を月に 1 回見る |
| バックアップ | `D:\webapp\backup\logs\` | backup.ps1 が 14 日より古い物を消す |
| Windows | イベント ビューアー（`eventvwr.msc`） | Windows が大きさの上限で管理する |

IIS のログを消すタスク（例）：

```
# 90 日より古い IIS のログを消すコマンド（長いので 2 つに分けて + でつなぐ）
$clean = 'Get-ChildItem C:\inetpub\logs\LogFiles -Recurse -Filter *.log | ' +
         'Where-Object { $_.LastWriteTime -lt (Get-Date).AddDays(-90) } | Remove-Item'
$action = New-ScheduledTaskAction -Execute "powershell.exe" -Argument "-NoProfile -Command `"$clean`""
Register-ScheduledTask -TaskPath "\webapp\" -TaskName "iis-log-cleanup" -Action $action `
  -Trigger (New-ScheduledTaskTrigger -Daily -At "03:00") `
  -Principal (New-ScheduledTaskPrincipal -UserId "SYSTEM" -LogonType ServiceAccount -RunLevel Highest)
```

> **用語:** アクセスログ（IIS が、来たお願いを 1 件 1 行で書く記録。いつ・どこから・どの URL に・どんな結果（ステータスコード）だったか）

> **用語:** イベント ビューアー（Windows 自身と、サービスの記録を見る画面。サービスが起動しなかった理由などが出る）

## 12.3 監視

「エラーを記録し、重大なエラーのときは IT 担当者へメールで知らせる」のは、アプリの機能です（設計例）。それに加えて、**アプリが止まっていることには、アプリ自身では気づけない**ので、外から見張ります。

| 何を見るか | どうやって | どれくらいの間隔 |
|---|---|---|
| アプリが返事をするか | 会社の監視の仕組み（あれば）か、タスク スケジューラで `http://webapp.example.local/health` を開く | 5 分ごとなど |
| サービスが動いているか | `Get-Service webapp-prod, webapp-stg, postgresql-x64-18, W3SVC` | 毎朝の点検 |
| ディスクの空き | `Get-Volume` | 週に 1 回。空きが 20% を切ったら、増やす相談をする |
| バックアップが成功したか | `Get-ScheduledTaskInfo` の `LastTaskResult` | 毎朝の点検 |

会社に監視の仕組み（Zabbix・System Center など）があれば、情報システム部門に、`/health` とサービスとディスクの見張りを頼むのが一番確かです。

> **用語:** 監視（システムが正しく動いているかを、決まった間隔で確かめ、おかしいときに人に知らせること）

> **用語:** W3SVC（IIS の、Web を受け付ける本体のサービスの名前。World Wide Web 発行サービス）

# 13. 動作確認・本番の開始・更新・切り戻し

## 13.1 検証環境で一通り確かめる

本番を始める前に、検証環境で、ログイン・依頼・承認のメール・図面の保存・問い合わせなど、主な使い方を一通り確かめます。確かめる項目は、07 付録の「動作確認のチェック表」にまとめます。

- 社員のパソコン（Edge と Chrome）から、`http://webapp-stg.example.local/` を開いて確かめます。
- 承認メールの URL を開くところまで確かめます（メールの URL の名前が、検証の名前になっているか）。
- 5MB に近い図面を保存できるか、10MB に近い添付ファイルを送れるかを確かめます（8.5 の上限）。
- サービスを止めて起動し直す、サーバーを再起動する、のあとに、自動で動き出すかを確かめます。

## 13.2 本番を始める

1. 検証と同じ手順で、本番を設定します（5.5 の本番の分、7 章、8 章の本番のサイトは済んでいる前提）。
2. 本番の社員データを取り込みます（7.7）。
3. 本番の URL（`http://webapp.example.local/`）を、社員に案内します。ほかのサービスと同じパスワードを使い回さないことも、あわせて案内します（00 基礎知識 6.2）。

## 13.3 更新の手順（2 回目以降）

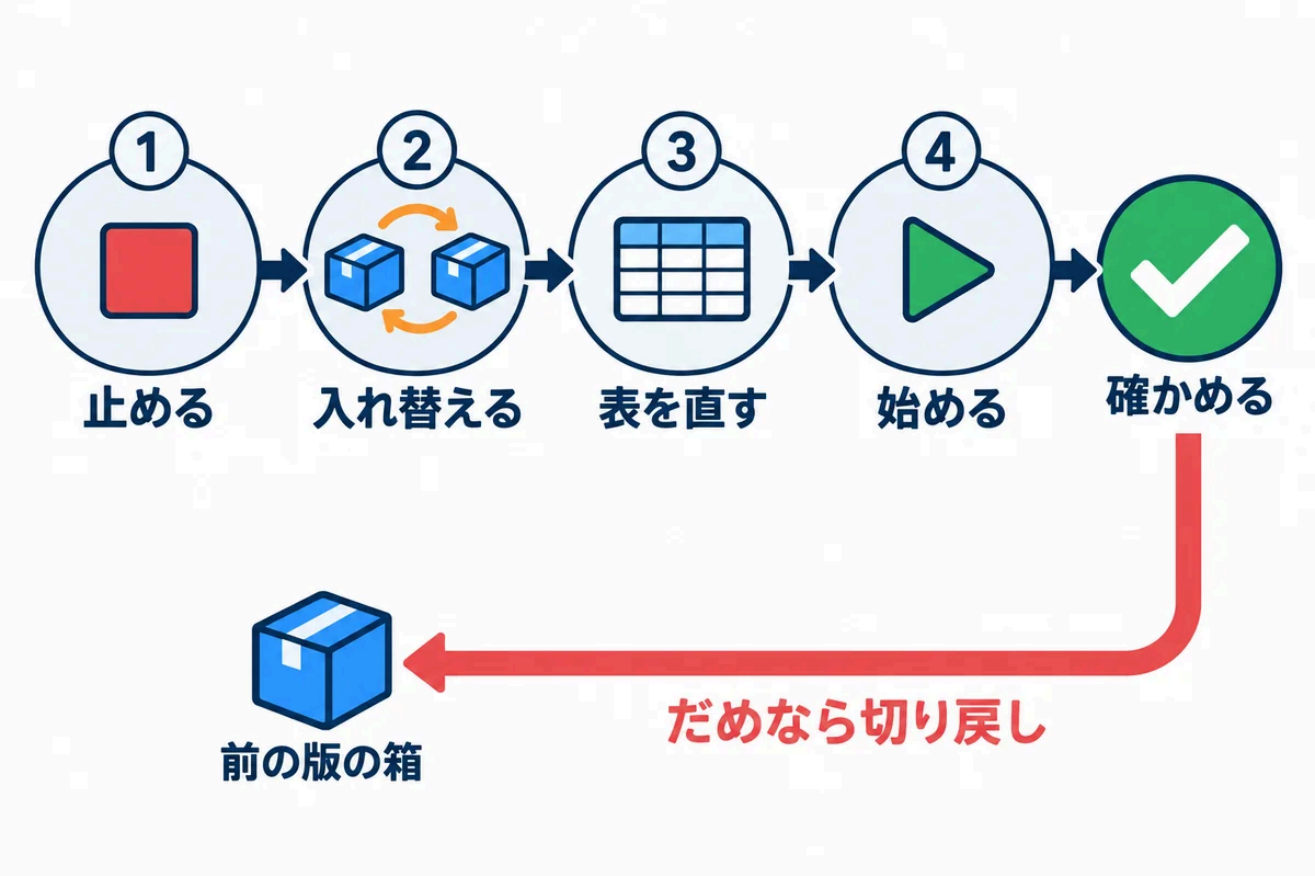



図13-1: 更新の 4 つの手順と、切り戻し

更新は、①検証で確かめる、②本番を止めてバックアップを取る、③入れ替える、④再開して確かめる、の 4 段で行います。例は、版 1.0.0 から 1.1.0 への更新です。

**① 検証で確かめる**

```
# 受け取った新しい版を確かめる
Get-FileHash D:\webapp\releases\webapp-1.1.0.zip -Algorithm SHA256

# 検証のアプリを入れ替える
Stop-Service webapp-stg
Rename-Item D:\webapp\stg\app app-prev-1.0.0
Expand-Archive D:\webapp\releases\webapp-1.1.0.zip -DestinationPath D:\webapp\stg\app
Start-Service webapp-stg
Invoke-WebRequest http://127.0.0.1:8081/health -UseBasicParsing | Select-Object StatusCode
```

検証で、変わった所と、主な画面（13.1 のチェック表）を確かめます。

**② 本番を止めて、バックアップを取る**

利用の少ない時間に、事前に社員へお知らせしてから行います。

```
Stop-Service webapp-prod
# 更新の前の控え（毎晩の控えとは別の名前で残す）
$env:Path += ";C:\Program Files\PostgreSQL\18\bin"
$env:PGPASSFILE = "D:\webapp\secrets\pgpass.conf"
pg_dump -h 127.0.0.1 -U webapp_prod_app -d webapp_prod -Fc -f "D:\webapp\backup\db\before_1.1.0_$(Get-Date -Format yyyyMMdd_HHmm).dump"
```

- アプリが止まっている間、社員のブラウザには、IIS の `502` のエラーが出ます（アプリから返事が無いため）。

**③ 入れ替える**

```
Rename-Item D:\webapp\prod\app app-prev-1.0.0
Expand-Archive D:\webapp\releases\webapp-1.1.0.zip -DestinationPath D:\webapp\prod\app
```

- テーブルの変更（マイグレーション）がある版は、この資料の設計例では、次の④で起動したときに、アプリが自動で当てます。

**④ 再開して確かめる**

```
Start-Service webapp-prod
Get-Service webapp-prod
Invoke-WebRequest http://127.0.0.1:8080/health -UseBasicParsing | Select-Object StatusCode
```

ブラウザで、ログインと主な画面を確かめます。問題が無ければ、更新の記録（いつ・どの版に・誰が）を残します。

## 13.4 切り戻し（前の版に戻す）

④で問題が見つかったら、**原因を探す前に、まず前の版に戻して**、社員が使える状態にします。

```
Stop-Service webapp-prod
Rename-Item D:\webapp\prod\app app-failed-1.1.0
Rename-Item D:\webapp\prod\app-prev-1.0.0 app

# テーブルの変更があった版のときは、②で取った控えに戻す
#  （古い版のアプリは、新しい形のテーブルを読めないことがあるため）
pg_restore -h 127.0.0.1 -U webapp_prod_app -d webapp_prod --clean --if-exists "D:\webapp\backup\db\before_1.1.0_（日時）.dump"

Start-Service webapp-prod
```

- データベースを控えに戻すと、②のあとに入った依頼や承認は消えます。②から④の間はアプリを止めているので、ふつうは消える物はありません。**本番を再開したあとで、時間がたってから戻すときは、消える物が出る**ので、関係者と相談してから戻します。
- `app-failed-1.1.0` は、原因を調べるために、しばらく残します。
- 落ち着いたら、使わなくなった古い `app-prev-…` のフォルダを消します（2〜3 版分は残しておくと安心です）。

> **用語:** 切り戻し（ロールバック：新しい版に問題があったとき、前の版に戻すこと。00 基礎知識 9.5）

# 14. セキュリティと日々の点検

## 14.1 建てたときに確かめること

| 項目 | 確かめ方 |
|---|---|
| アプリと DB が 127.0.0.1 だけで待ち受けている | 9.3 |
| `D:\webapp` を、管理者・SYSTEM・アプリのアカウントだけがさわれる | `icacls D:\webapp`・`icacls D:\webapp\prod` |
| パスワードが書かれたファイル（WinSW の XML・pgpass）を、ほかの人が読めない | `icacls`（7.4・11.2） |
| 社員データの CSV が、サーバーやパソコンに残っていない | `D:\webapp\installers\` などを見る |
| アプリが仮想アカウントで動いている（管理者ではない） | 7.4 の `Get-CimInstance Win32_Service` |
| スーパーユーザー `postgres` のパスワードを、会社の決まりに従って保管した | 自分で確かめる |
| ウイルス対策ソフトの除外 | PostgreSQL のデータのフォルダを、ウイルス対策の検査から外すかどうかを、情報システム部門と決める（検査がデータベースのファイルを掴むと、遅くなったり壊れたりすることがあるため） |

## 14.2 日々・月々の点検（例）

| いつ | 何をするか |
|---|---|
| 毎朝 | サービス 4 つが動いているか、夜のバックアップが成功したか（12.3） |
| 毎週 | ディスクの空き、アプリのログにエラーが続いていないか |
| 毎月 | Windows Update（会社の決まりに従う）、IIS・PostgreSQL のログのたまり具合 |
| 3 か月ごと | リストアの練習（11.5） |
| PostgreSQL の修正版が出たとき | 18.x の修正版（例：18.6 → 18.7）を、検証環境で確かめてから入れる。修正版の入れ替えは、同じインストーラーを上から入れる方法でできる（データはそのまま） |
| 年に 1 回 | パスワードの見直し、この資料の手順が今のサーバーと合っているか、HTTPS にできるようになったか |

- PostgreSQL 18 のサポートは 2030 年 11 月まで、と案内されています。18 → 19 のような大きな版の上げ方は、データの移しかえ（`pg_upgrade` か、`pg_dump` と `pg_restore`）が要るので、別に計画します。

## 15. 困ったとき（トラブルシューティング）

まず、**どこで止まっているか**を、手前から順に切り分けます。

1. 社員のパソコン → サーバーの 80 番（DNS・ネットワーク・ファイアウォール）
2. IIS → アプリの 8080 番（IIS の設定・アプリのサービス）
3. アプリ → データベース・ファイル保存フォルダ・メール

| 見えること | 考えられる原因 | 確かめること |
| --- | --- | --- |
| ブラウザで「このサイトにアクセスできません」 | DNS に名前が無い、80 番が通っていない、IIS が止まっている | `Resolve-DnsName`・`Test-NetConnection -Port 80`（8.6）、`Get-Service W3SVC` |
| `502.3`（Bad Gateway）と出る | アプリが止まっている、ポートが違う | `Get-Service webapp-prod`、`Invoke-WebRequest http://127.0.0.1:8080/health`、web.config の行き先 |
| IIS の `404` が出て、アプリの画面にならない | URL Rewrite か ARR が入っていない、取り次ぎが無効、web.config の置き場所が違う | 8.2・8.3、`D:\webapp\prod\iis\web.config` があるか |
| ファイルを送ると `404.13` | 送る大きさが IIS の上限を超えた | 8.5 の `maxAllowedContentLength` |
| ログイン・保存などのボタンを押すと `403`（本文が forbidden だけ） | Host が書きかえられて、Topcoat の OriginPolicy が止めた | 8.3 の `preserveHostHeader` |
| ログインしても、次の画面でログインしていない扱いになる | セッションのクッキーに `Secure` が付いていて、HTTP のブラウザが受け取らない | 6.4、ブラウザの開発者ツール（F12）の「アプリケーション → Cookie」 |
| サービスが起動しない、すぐ止まる | XML の書き間違い、`assets\` が無い、許可が足りない、DB につなげない | `D:\webapp\prod\logs\` の `wrapper.log` と `err.log`、イベント ビューアーの「Windows ログ → システム」 |
| バックアップのタスクの「前回の実行結果」が `(0x1)` で、`D:\webapp\backup\logs\` に記録も無い | `backup.ps1` を BOM 無しの UTF-8 で保存したため、PowerShell 5.1 が日本語の所で読み違えて、構文エラーで動いていない | 11.3 の「UTF-8 (BOM 付き)」で保存し直す。`powershell -File D:\webapp\tools\backup.ps1` を手で動かすと、構文エラーの文が見える |
| 「エラー 1053」 | 実行ファイルを WinSW で包まずに、直接サービスにした | 7.1・7.4 |
| アプリのログに、データベースにつなげないエラー | PostgreSQL が止まっている、パスワードが違う、`pg_hba.conf` | `Get-Service postgresql-x64-18`、5.6 の `psql` |
| 添付ファイルを保存できない | `files\` に書く許可が無い、ディスクがいっぱい | 7.4 の `icacls`、`Get-Volume` |
| メールが届かない | 25 番が通らない、コネクタ、SPF | 10.2・10.3 の表 |
| 時刻が 9 時間ずれる | タイムゾーンが UTC のまま | 3.4、5.3 の `timezone` |
| 再起動のあと、アプリだけ動いていない | PostgreSQL より先に起動しようとした、起動の種類が手動 | 7.3 の `<depend>`、`Get-CimInstance Win32_Service` の `StartMode` |

# 16. 用語ミニ辞典（この資料に出た主な言葉）

| 用語 | 短い説明 | 章 |
|---|---|---|
| appcmd | IIS の設定をコマンドで変える道具 | 8.3 |
| ARR | IIS が受けたお願いを、アプリに取り次ぐ部品 | 8.2 |
| Cargo.lock | アプリが使う部品の正確な版の記録 | 6.1 |
| CSRF | ほかのサイトから、勝手に操作を送らせる攻撃 | 8.3 |
| CSV | 表のデータをカンマで区切った文字のファイル | 7.7 |
| DLL | 実行ファイルが使う部品のファイル | 6.2 |
| gMSA | パスワードを AD が管理するサービス用アカウント | 4.3 |
| listen_addresses | PostgreSQL が待ち受ける住所の設定 | 5.3 |
| pg_dump・pg_restore | PostgreSQL の控えを取る道具・戻す道具 | 11.1・11.5 |
| pg_hba.conf | PostgreSQL に誰がどこからつないでよいかの決まり | 5.3 |
| pgpass | パスワードを打たずにつなぐためのファイル | 11.2 |
| psql | PostgreSQL に SQL を打ち込む道具 | 5.4 |
| release ビルド | 本番用に、速く動くように作ること | 6.1 |
| robocopy | フォルダを丸ごと写す Windows の道具 | 11.1 |
| Secure・__Host-・HttpOnly | クッキーに付ける印（HTTPS だけ・名前の決まり・JavaScript から読めない） | 6.4 |
| SID | アカウントやグループの、世界で1つの番号 | 4.2 |
| URL Rewrite | 来た URL を決まりに従って渡す IIS の部品 | 8.2 |
| web.config | IIS のサイトごとの設定ファイル | 8.5 |
| WinSW | ふつうの実行ファイルを Windows サービスにする道具 | 7.1 |
| アクセス許可（ACL）・継承 | 誰が何をしてよいかの一覧・上のフォルダからの引き継ぎ | 4.2 |
| 仮想アカウント | サービスごとにできる、パスワードの無い専用のアカウント | 4.3 |
| 切り戻し | 前の版に戻すこと | 13.4 |
| 終了コード | プログラムが終わるときに返す番号 | 11.3 |
| 静的リンク | 部品を実行ファイルの中に入れてしまうこと | 6.2 |
| 設計例 | この資料で「こう作る」と決めた一つの形 | はじめ |

| 用語 | 短い説明 | 章 |
| --- | --- | --- |
| 接続文字列 | データベースにつなぐ情報を1行にしたもの | 7.3 |
| ハッシュ値 | ファイルの中身から計算した指紋 | 5.1 |
| バインド | サイトと IP・ポート・名前の結び付け | 8.4 |
| ヘルスチェック | アプリに「動いていますか」と聞く入口 | 7.5 |
| ロール・OWNER | PostgreSQL のユーザー・データベースの持ち主 | 5.5 |